STUDY GUIDE

.NET Tech Lead

DSA in C# and system design

Coding screen · Whiteboard · ASP.NET Core · Azure SQL · Service Bus

For a developer with about 8 years of .NET who is moving into a tech-lead interview and needs a practical DSA refresh in C#.

Prepared 31 October 2026. Examples target C# 14 on .NET 10 LTS, supported through 14 November 2028. This is a companion to the 10-day roadmap, the deep dive, and the 20-day pipeline and MVC guide.

How this guide sits with the others

You already have a spoken script and a worked example for the backend you have shipped. The 10-day roadmap is that script: C#, SOLID, the Web API, EF Core, SQL, Azure App Service and Functions, service boundaries, migration, and a leadership mock. The deep dive is the explanation behind the script. The 20-day pipeline and MVC guide is the longer pass over middleware, filters, model binding, and how a controller action actually runs. This PDF is the other two rooms.

Part A is the coding screen. You get a shared editor, a problem you have seen the shape of, and a person watching how you clarify, test, and talk about complexity. It is a practical refresh in C#, using List, Dictionary, HashSet, Queue, Stack, and PriorityQueue. It is not a competitive-programming camp, and it is not a beginner tutorial. You will not study segment trees. You will finish a correct solution, name the cost, and hit the C# mistakes that show up when an eight-year developer is rusty.

Part B is the whiteboard. You design a .NET backend out loud in about 20 minutes, and on the last day you hold a 45-minute mock. The stack is the one you already speak: ASP.NET Core, EF Core, SQL Server or Azure SQL, Azure App Service, Azure Functions on the isolated worker, Azure Service Bus, Microsoft Entra ID, and managed identity. When a design needs a second deployable, you name the boundary and stop. The microservices chapter in the roadmap and the deep dive is where you practice the cut. This guide does not teach that chapter again.

Guide	Room it prepares	What you do with it
10-day roadmap	The conversation	90-second answers, stories, and the stack you name.
Deep dive	The follow-up	The worked example behind each roadmap day.
20-day pipeline and MVC	The request path	Middleware, filters, binding, and the action.
This PDF, Part A	The coding screen	Ten patterns in C#, each with a drill.
This PDF, Part B	The whiteboard	Ten designs, anchored on the order API.

Numbers in this guide

Where a number is a load or a latency target, it is an assumption you say out loud so the panel can correct it. The running order system assumes about 50 requests a second at peak, about 2 million orders a year, a read p95 under 300 milliseconds, and email within a minute. Other days label their own assumptions. Nothing in here is an Azure quota, a price, or a SKU limit. When a platform limit would change the design, say which document you would open.

How to spend a day

Each day is about 3.5 hours. Reading the page quietly does not change the interview. Part A wants the code in your hands. Part B wants the design on paper, then spoken against a clock.

1. Part A: trace the worked example, close the page, write it from memory, then write the variant.
2. Say the complexity and the C# pitfall before you look back.
3. Speak the Say this box in under 90 seconds.
4. Part B: draw the spine, write the sketch, then give the 20-minute version twice.
5. Keep a console project on the runtime you will interview in. A single file and a Main method are enough.

The twenty days

Day	Part A — coding screen	Part B — whiteboard
1	Arrays, strings, hash maps	The design spine
2	Two pointers and sliding window	The order API, taught fully
3	Stack and queue	Catalog reads and order writes
4	Linked lists	Nightly million-row ingest
5	Binary search	Multi-tenant SaaS
6	Trees and BST	Notification fan-out
7	Heaps and PriorityQueue	Rate limiting and the public gateway
8	Graphs	A URL shortener
9	Recursion and backtracking	Realtime order status
10	Dynamic programming, and how you talk	A 45-minute mock

C# pitfalls you will hit in the room

These five show up across the ten days. Name the one that applies before the interviewer does.

Pitfall	What it looks like	What you do
int overflow	A sum, a midpoint, or a Fibonacci total wraps in the default unchecked context.	Use long for sums and running totals. Compute a midpoint as lo + (hi - lo) / 2.
Missing dictionary key	dict[key] or dict[key]++ throws KeyNotFoundException.	TryGetValue, GetValueOrDefault, or HashSet.Add.
Unstable sort	Array.Sort and List<T>.Sort may reorder equal keys.	Keep the payload inside the sorted item. When ties must keep input order, use LINQ OrderBy, which is stable.
Recursion depth	A skewed tree, a long linked list, or a deep graph DFS throws StackOverflowException. C# does not promise tail-call elimination.	Write the loop or an explicit Stack when the input can be deep. Subsets of a small n are fine.
Off-by-one	A window length, a half-open range, or a binary-search bound is one slot wrong.	Say the invariant out loud. Window length is right - left + 1. A half-open range is [lo, hi).

Say this

I treat the coding screen as a C# problem with a complexity I can defend, and the whiteboard as a .NET service with a transaction, a failure, and a rollback. I label every load number as an assumption. I do not invent an Azure quota.

Part A — DSA in C# for the coding screen

Ten days, one pattern a day. The code is the code you write in a console app, not pseudocode. Assume the usings you already know: System, System.Collections.Generic, and System.Linq. Each drill is two problems: the worked example from memory, then the variant. Close this page before you type.

Part A, Day 1 — Arrays, strings, and hash maps

Outcome. You can solve a frequency problem, an anagram problem, and a two-sum problem in one pass, and you can say what the dictionary costs. You can also say when a sort is the wrong tool because it is unstable or because it throws the original index away.

Time box. About 3.5 hours. Spend 30 minutes tracing the three methods on paper. Spend 45 minutes rewriting them from memory. Spend 30 minutes on complexity and the two pitfalls, missing keys and overflow. Spend 45 minutes on the variant. Spend the rest comparing, then speaking the Say this box in under 90 seconds.

The idea

An array gives you O(1) indexing and no fast membership test. A Dictionary or a HashSet turns a membership or a frequency question into expected O(1) per element, at the cost of extra memory and a worst case you do not pretend is impossible. You reach for the map when the question is: have I seen this value, how many times, or what index had the complement.

Two-sum is the shape. For each number, the complement is target minus that number. If the complement is already in the map, you have the pair and the earlier index. If not, you store this number and its index. One pass, expected linear time, linear memory. The brute force pair of loops is n squared and is what you say first so they hear you can start.

Anagrams are a frequency problem. Two strings are anagrams when they have the same length and the same character counts. A group of anagrams shares a key. If the problem promises lowercase English letters, a count of 26 is the key. The counts need a delimiter when you join them into a string, because 1 followed by 11 is the same text as 11 followed by 1. If the alphabet is open, sort the characters and use the sorted string as the key. Sorting characters does not care about sort stability. Stability matters later, when equal business keys must keep their input order.

Worked example

```
public static int[] TwoSum(int[] nums, int target)
{
    var seen = new Dictionary<int, int>(nums.Length);
    for (int i = 0; i < nums.Length; i++)
    {
        int need = target - nums[i];
        if (seen.TryGetValue(need, out int j))
```

```
            return new[] { j, i };
        seen[nums[i]] = i;
    }
    return Array.Empty<int>();
}

public static bool AreAnagrams(string left, string right)
{
    if (left.Length != right.Length)
        return false;

    var counts = new Dictionary<char, int>();
    foreach (char c in left)
        counts[c] = counts.GetValueOrDefault(c) + 1;

    foreach (char c in right)
    {
        if (!counts.TryGetValue(c, out int n) || n == 0)
            return false;
        counts[c] = n - 1;
    }
    return true;
}

// Assumption for this key: the word is lowercase a-z.
public static List<List<string>> GroupAnagrams(string[] words)
{
    var groups = new Dictionary<string, List<string>>();
    foreach (string word in words)
    {
        string key = AnagramKey(word);
        if (!groups.TryGetValue(key, out var list))
        {
            list = new List<string>();
            groups[key] = list;
        }
        list.Add(word);
    }
    return groups.Values.ToList();
}

static string AnagramKey(string word)
{
    Span<int> counts = stackalloc int[26];
    foreach (char c in word)
        counts[c - 'a']++;

    var sb = new StringBuilder(26 * 4);
    for (int i = 0; i < counts.Length; i++)
    {
        sb.Append(counts[i]);
        sb.Append('#');
    }
```

```
    return sb.ToString();
}
```

Trace two-sum on [2, 7, 11, 15] with target 9. At 2 the map is empty, so you store 2 at index 0. At 7 the complement is 2, which is in the map, so the answer is indices 0 and 1. You store a value after the lookup, so a value is not paired with itself unless it appears twice.

Span is doing a small job here: the 26 counts live on the stack, so you do not allocate an array per word. The string key is still an allocation, and that is fine. If the interviewer wants less allocation and the groups are small, say so and keep the clear version. Do not start the interview inside stackalloc.

Complexity

Two-sum is expected O(n) time and O(n) extra memory. Anagram compare is O(n) time and O(alphabet) memory. Grouping m words of length k with a 26-count key is O(m k) time. The sort-the-characters key is O(m k log k) and is the right key when the alphabet is not a fixed 26. Hashing is expected constant time per operation. You say expected, because a pathological hash is a conversation you will not pretend you have closed.

The sort you did not use

You can sort pairs of (value, index) and walk inward. That is O(n log n) and it keeps the index because the index rides inside the tuple. It is a worse two-sum, and it is a useful backup if you blank on the map. Array.Sort on those tuples is not stable, and you do not care, because the index is part of the value. The day stability matters is a list of orders sorted by priority where two orders share a priority and the original order is the tie-break. Array.Sort and List<T>.Sort may reorder those equals. Enumerable.OrderBy is stable. Say that if they ask you to sort.

target - nums[i] is an int subtraction. In the default unchecked context it wraps. The usual interview input fits. If the problem says values can sit at the ends of the int range, compute the complement in a long and say why.

Interview trap

dict[c]++ throws when c is missing. GetValueOrDefault or TryGetValue is the fix. A second trap is using the sorted position as if it were the original index. A third is an anagram key with no delimiter between counts. A fourth is calling two strings anagrams after sorting them with a culture-sensitive compare. For a machine key, ordinal is the compare you want.

Say this

One pass, a dictionary of value to index, TryGetValue so a miss is not an exception. Expected linear time and linear memory. I hash the raw character counts when the alphabet is fixed, and I only sort when I need a key for an open alphabet or when I need order. If equal keys must keep their input order, I use OrderBy because Array.Sort is not stable.

Drill

1. Close the page. Write TwoSum and AreAnagrams from memory, including the empty return and the length check.
2. Variant: first non-repeating character. One pass to count, a second pass to return the first character whose count is 1. Return a nullable char when there is none.
3. State time, memory, and the missing-key pitfall before you look back.

Variant, after you have written it

```
public static char? FirstUnique(string s)
{
    var counts = new Dictionary<char, int>();
    foreach (char c in s)
        counts[c] = counts.GetValueOrDefault(c) + 1;

    foreach (char c in s)
    {
        if (counts[c] == 1)
            return c;
    }
    return null;
}
```

The second loop may use counts[c] because every character was inserted in the first loop. That is the difference between a key you know exists and a key you hope exists. Time O(n), memory O(alphabet).

Part A, Day 2 — Two pointers and sliding window

Outcome. You can move two indices toward each other on a palindrome or a sorted pair, and you can grow and shrink a window over a contiguous range. You compute the length as right - left + 1 without hesitating.

Time box. About 3.5 hours. Trace both methods on a short string and a short array. Rewrite them from memory. Then do the fixed-window variant. Leave 20 minutes to say the invariant out loud: what the window contains after each move of left.

The idea

Two pointers apply when you can decide, from the values at the ends, which end to move. A palindrome compares ends and walks inward. A sorted array looking for a pair with a given sum moves the left end up when the sum is too small and the right end down when the sum is too large. You do not use two pointers on an unsorted array and then claim the pair is found, unless you sorted a copy first and paid n log n.

A sliding window applies when the answer is a contiguous subarray or substring, and growing the right end only adds information you can undo by moving the left end. The window is valid, or it is not, and you always know which. For a longest substring without a repeated character, the window contains unique characters. When the next character is already inside, you shrink from the left until it is not. For a minimum length subarray whose sum reaches a target, the numbers need to be non-negative so that growing the right end only increases the sum and shrinking the left end only decreases it. Negative numbers break that monotone move, and you say so instead of forcing the pattern.

Worked example

```
public static bool IsPalindrome(ReadOnlySpan<char> s)
{
    int i = 0;
    int j = s.Length - 1;
    while (i < j)
    {
        while (i < j && !char.IsLetterOrDigit(s[i]))
            i++;
        while (i < j && !char.IsLetterOrDigit(s[j]))
            j--;
        if (char.ToLowerInvariant(s[i]) != char.ToLowerInvariant(s[j]))
            return false;
        i++;
        j--;
    }
    return true;
}

public static int LongestUnique(string s)
{
    var inWindow = new HashSet<char>();
    int left = 0;
```

```
    int best = 0;
    for (int right = 0; right < s.Length; right++)
    {
        while (!inWindow.Add(s[right]))
        {
            inWindow.Remove(s[left]);
            left++;
        }
        int length = right - left + 1;
        if (length > best)
            best = length;
    }
    return best;
}

public static int MinSubarrayLen(int[] nums, int target)
{
    int left = 0;
    long sum = 0;
    int best = int.MaxValue;
    for (int right = 0; right < nums.Length; right++)
    {
        sum += nums[right];
        while (sum >= target && left <= right)
        {
            int length = right - left + 1;
            if (length < best)
                best = length;
            sum -= nums[left];
            left++;
        }
    }
    return best == int.MaxValue ? 0 : best;
}
```

ReadOnlySpan<char> lets the palindrome walk the string without building a cleaned copy. Indexing a span is the same as indexing the string. This is a justified use of Span: it removes an allocation without hiding the algorithm. A string is UTF-16, so s[i] is a code unit, not a grapheme. For the usual interview alphabet that is enough, and you mention it if they hand you emoji or combining marks.

Trace LongestUnique on abcabcbb. The window grows to abc. At the second a, Add fails, you remove from the left until a is gone, and the window becomes bca. The best length is 3. HashSet.Add returns false when the character was already present, which is the test and the insert in one call.

MinSubarrayLen keeps the sum in a long. An int sum of a large array wraps in the unchecked context and the window then shrinks or grows for the wrong reason. The problem statement for this pattern assumes non-negative numbers. Say that assumption before you code.

Complexity

The palindrome is O(n) time and O(1) extra memory. LongestUnique is O(n) time because each index enters and leaves the window at most once, and O(alphabet) memory. MinSubarrayLen is O(n) time and O(1) extra

memory on non-negative numbers. A window that shrinks from the inside, or a sum that is allowed to go backwards because of negatives, is no longer this pattern.

Interview trap

The length is right - left + 1, not right - left. The inner while on the palindrome must also keep i < j, or you walk off the string. Moving only the right pointer and restarting left at zero on every duplicate is still correct and no longer linear. An int running sum is the overflow. Two pointers on unsorted data is a guess.

Say this

Ends move inward when a comparison tells me which end is wrong. A window grows on the right and shrinks on the left while I can state what the window means. Length is right minus left plus one. I keep the sum in a long, and I only use this window when adding on the right moves the sum in one direction.

Drill

1. Write IsPalindrome and LongestUnique from memory.
2. Variant: maximum sum of a fixed window of size k. One pass, add the new element, subtract the element that fell out, track the best sum in a long.
3. Say what you return for an empty array and for k larger than the array.

Variant, after you have written it

```
public static long MaxSumFixed(int[] nums, int k)
{
    if (k <= 0 || k > nums.Length)
        return 0;

    long sum = 0;
    for (int i = 0; i < k; i++)
        sum += nums[i];

    long best = sum;
    for (int right = k; right < nums.Length; right++)
    {
        sum += nums[right];
        sum -= nums[right - k];
        if (sum > best)
            best = sum;
    }
    return best;
}
```

Fixed windows do not need a left pointer if you can name the element that left: it is at right - k. Time O(n), extra memory O(1). Returning 0 for a bad k is a choice; in the interview, ask what they want for k = 0.

Part A, Day 3 — Stack and queue

Outcome. You can match brackets with a stack, explain a monotonic stack on the next-greater problem, and use a queue for breadth-first search so the first visit is the shortest path in an unweighted graph.

Time box. About 3.5 hours. Code the brackets until the leftover-open case and the early-close case are both tests you remember. Then the next-greater loop. Then a tiny graph BFS. The variant is the daily-temperatures form of the same monotonic stack.

The idea

A stack is last in, first out. It remembers the thing you opened most recently, which is exactly what nested brackets need. You push opens. On a close, the stack must be non-empty and the pop must be the matching open. At the end the stack must be empty. ([)] fails because the close does not match the latest open, even though the counts of each bracket would match.

A monotonic stack keeps indices whose values are still waiting for an answer, in order. For the next greater element you keep a stack of indices whose values are decreasing. When the new value is greater than the value at the top, that top has found its answer, you pop it, and you keep popping while the new value is still greater. Each index is pushed once and popped once.

A queue is first in, first out. Breadth-first search uses it. You mark a node seen when you enqueue it, not when you dequeue it, or the same neighbor gets enqueued many times. In an unweighted graph the first time you reach a node is a shortest path. Depth-first search finds a path. It does not find the shortest one. Use System.Collections.Generic.Stack<T> and Queue<T>. The non-generic Stack and Queue are the old untyped collections.

Worked example

```
public static bool IsValidBrackets(string s)
{
    var stack = new Stack<char>();
    foreach (char c in s)
    {
        if (c is '(' or '[' or '{')
        {
            stack.Push(c);
            continue;
        }

        if (stack.Count == 0)
            return false;

        char open = stack.Pop();
        bool match = (open, c) is ('(', ')') or ('[', ']') or ('{', '}');
        if (!match)
            return false;
    }
```

```
    return stack.Count == 0;
}

public static int[] NextGreater(int[] nums)
{
    var result = new int[nums.Length];
    Array.Fill(result, -1);
    var stack = new Stack<int>();
    for (int i = 0; i < nums.Length; i++)
    {
        while (stack.Count > 0 && nums[stack.Peek()] < nums[i])
            result[stack.Pop()] = nums[i];
        stack.Push(i);
    }
    return result;
}

public static int ShortestPath(Dictionary<int, List<int>> graph, int start, int goal)
{
    var queue = new Queue<int>();
    var dist = new Dictionary<int, int> { [start] = 0 };
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        int node = queue.Dequeue();
        if (node == goal)
            return dist[node];

        if (!graph.TryGetValue(node, out var nexts))
            continue;

        foreach (int next in nexts)
        {
            if (dist.ContainsKey(next))
                continue;
            dist[next] = dist[node] + 1;
            queue.Enqueue(next);
        }
    }
    return -1;
}
```

Trace brackets on ([)]. Push '(', push '[', then ')' does not match '[', so the method returns false. Trace ()[] : each close matches the latest open, and the stack ends empty. Trace ((: the loop ends with two opens still on the stack, so the method returns false.

NextGreater on [2, 1, 2, 4, 3] yields [4, 2, 4, -1, -1]. The strict < means an equal neighbor is not a greater element. If the problem wants the next greater or equal, you change the comparison and you say you changed it. The stack stores indices, not values, so you can write the answer back into the right slot.

ShortestPath marks distance at enqueue time, which is also the seen mark. TryGetValue covers a node that has no outgoing list, so a missing key is a dead end and not an exception. The distance is the edge count. This

graph is unweighted. A weighted graph wants a priority queue, which is Day 7's tool, and you do not pretend BFS still gives the shortest path once edges have costs.

Complexity

Brackets are O(n) time and O(n) stack memory. Next greater is O(n) time because each index is pushed and popped at most once, and O(n) memory. BFS is O(n + e) time for n nodes and e edges, and O(n) memory for the queue and the distance map. Building the adjacency list is part of the cost. Say it.

Interview trap

A count of opens and closes accepts ([)]. Forgetting the final empty-stack check accepts ((. Using a stack for BFS changes the visit order and loses the shortest-path claim. Marking a node seen on dequeue lets the queue hold the same node many times. dict[node] on a node with no edge list throws.

Say this

A stack matches the latest open. A monotonic stack of indices answers the next greater element in one pass, each index pushed once. A queue is breadth-first, and in an unweighted graph the first visit is the shortest path. I mark the node when I enqueue it.

Drill

1. Write IsValidBrackets and NextGreater from memory. Include the tests (), ([)], and ((.
2. Variant: daily temperatures. Given an array of temperatures, return how many days until a warmer day, or 0. Same decreasing stack of indices. The answer is i - poppedIndex, not the temperature.
3. Say why the stack stores the index.

Variant, after you have written it

```
public static int[] DaysUntilWarmer(int[] temps)
{
    var wait = new int[temps.Length];
    var stack = new Stack<int>();
    for (int i = 0; i < temps.Length; i++)
    {
        while (stack.Count > 0 && temps[stack.Peek()] < temps[i])
        {
            int earlier = stack.Pop();
            wait[earlier] = i - earlier;
        }
        stack.Push(i);
    }
    return wait;
}
```

The array starts as zeros, so an index that never pops stays 0. Time O(n), memory O(n).

Part A, Day 4 — Linked lists

Outcome. You can reverse a singly linked list without losing the tail, and you can detect a cycle by reference identity. You know why the node type is a class.

Time box. About 3.5 hours. Draw a four-node list and rewire it with a pencil before you code. Then write reverse from memory until you can do it without the drawing. Then Floyd. The variant is the start of the cycle. Leave time to explain the record trap out loud.

The idea

The interview list is a node type you control, with a value and a Next reference. LinkedList<T> in the BCL is a doubly linked list with a different API. You can mention it. You do not use it to answer reverse, because the point of the question is the rewiring.

Reverse walks once. You keep three references: previous, current, and the old next. You must copy current.Next into a local before you overwrite it, or the rest of the list is gone. Then you point current.Next at previous, advance previous to current, and advance current to the saved next. When current is null, previous is the new head. The empty list and the single-node list both fall out of the same loop.

A cycle means a node's Next eventually points at an earlier node. Floyd uses two pointers, slow moving one step and fast moving two. If there is no cycle, fast reaches the end. If there is a cycle, fast laps slow and they meet. You compare references, not values. Two nodes can hold the same integer and not be a cycle.

Make the node a sealed class. A record compares by value. If you write slow == fast on a record, equality walks the chain, which is the wrong question and can blow the stack on a cycle. ReferenceEquals asks the question you meant. A recursive reverse is easy to write and uses one stack frame per node. A long list can throw StackOverflowException. C# does not promise it will turn that recursion into a loop. The iterative reverse is the one you ship in the interview.

Worked example

```
public sealed class ListNode
{
    public int Value;
    public ListNode? Next;
    public ListNode(int value) => Value = value;
}

public static ListNode? Reverse(ListNode? head)
{
    ListNode? previous = null;
    ListNode? current = head;
    while (current is not null)
    {
        ListNode? next = current.Next;
        current.Next = previous;
        previous = current;
```

```
        current = next;
    }
    return previous;
}

public static bool HasCycle(ListNode? head)
{
    ListNode? slow = head;
    ListNode? fast = head;
    while (fast?.Next is not null)
    {
        slow = slow!.Next;
        fast = fast.Next.Next;
        if (ReferenceEquals(slow, fast))
            return true;
    }
    return false;
}
```

Trace 1 -> 2 -> 3. Start with previous null and current 1. Save 2, point 1 at null, previous becomes 1, current becomes 2. Save 3, point 2 at 1, previous becomes 2, current becomes 3. Save null, point 3 at 2, previous becomes 3, current becomes null. Return 3, which now points at 2, which points at 1, which points at null.

The ! on slow is there because the compiler does not connect the loop condition on fast to slow. On a well-formed list slow is non-null whenever fast.Next is non-null. If you would rather avoid the operator, assign slow to a local and return false when it is null. That branch does not run for a real list.

Complexity

Reverse is O(n) time and O(1) extra memory. Cycle detection is O(n) time and O(1) extra memory. A HashSet of seen nodes is also correct, and it uses O(n) memory. Give Floyd when they ask for constant memory. Give the set when you want the simpler code and memory is free. Say which one you wrote.

Interview trap

Overwriting Next before saving it loses the list. Comparing Value finds duplicate numbers, not a cycle. A record with == walks values. A recursive reverse on a long list is a stack overflow. Forgetting to return previous returns null, because current ends null. The BCL LinkedList<T> is not the answer they asked for.

Say this

I keep the old Next in a local, then point the node at previous. The new head is previous when current runs off the end. A cycle is two pointers and ReferenceEquals, on a class, in constant extra memory. A HashSet of nodes is the linear-memory version. I do not recurse down a list that can be long.

Drill

1. Write the node class, Reverse, and HasCycle from memory. Draw the four pointers on a four-node list after each iteration.

2. Variant: return the node where the cycle begins, or null when there is no cycle. After slow and fast meet, walk one pointer from the head and one from the meeting point, one step at a time. They meet at the start.
3. State the memory of Floyd versus the memory of a seen set.

Variant, after you have written it

```
public static ListNode? CycleStart(ListNode? head)
{
    ListNode? slow = head;
    ListNode? fast = head;
    while (fast?.Next is not null)
    {
        slow = slow!.Next;
        fast = fast.Next.Next;
        if (!ReferenceEquals(slow, fast))
            continue;

        ListNode? fromHead = head;
        while (!ReferenceEquals(fromHead, slow))
        {
            fromHead = fromHead!.Next;
            slow = slow!.Next;
        }
        return fromHead;
    }
    return null;
}
```

You can state the meeting rule without deriving the algebra in the interview. If they ask why it works: the meeting point is some number of steps into the cycle, and walking from the head covers the prefix in the same number of steps the pointer inside the cycle needs to finish a lap back to the start. Time O(n), extra memory O(1).

Part A, Day 5 — Binary search

Outcome. You can write a binary search with a half-open range that terminates, and you can binary-search the answer when the array is not what you are indexing. You can say the feasibility predicate in one sentence.

Time box. About 3.5 hours. Write the lower-bound loop until the invariant is boring. Then the ship-capacity problem, which is the answer-range form. The variant is the upper bound, the first index whose value is strictly greater than the target. Spend the last half hour on off-by-one: change one bound the wrong way and watch the loop stick, so you remember why you will not mix the two loop styles.

The idea

Binary search needs a monotone predicate. On a sorted array the predicate is: this index is too far left, or it is a candidate. The version you should be able to write half asleep uses a half-open interval [lo, hi). lo starts at 0. hi starts at a.Length, not a.Length - 1, because the insertion point might be past the last element. While lo is less than hi, the midpoint is lo + (hi - lo) / 2. Integer division makes that midpoint strictly below hi, so the range shrinks. If a[mid] is less than the target, the answer is in (mid, hi), so lo becomes mid + 1. Otherwise the answer is in [lo, mid], so hi becomes mid. When the loop ends, lo is the first index whose value is greater than or equal to the target, or a.Length if every value is smaller.

lo + (hi - lo) / 2 avoids the overflow of (lo + hi) / 2. Two large indices can sum past Int32.MaxValue. Array lengths can be large enough for that sum to wrap. The wrapped midpoint is nonsense, and the loop can run forever or throw. This is the overflow pitfall in its interview form.

Array.BinarySearch exists. It returns the index, or the bitwise complement of the insertion point when the value is missing. You may mention it. You still write the loop, because they want the invariant, and because the complement trick is easy to get wrong under pressure.

Searching on the answer is the same loop with a different predicate. The classic form here is: what is the smallest truck capacity that ships every package, in order, within D days. Capacity is monotone. If a capacity works, every larger capacity works. If it fails, every smaller capacity fails. lo is the heaviest single package, because a package cannot be split. hi is the sum of the packages, which ships in one day. You binary-search that range. The predicate walks the array once and counts the days. The array is not sorted. The search space is.

Worked example

```
public static int LowerBound(int[] a, int target)
{
    int lo = 0;
    int hi = a.Length; // half-open [lo, hi)
    while (lo < hi)
    {
        int mid = lo + ((hi - lo) / 2);
        if (a[mid] < target)
```

```
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}

public static int MinShipCapacity(int[] weights, int maxDays)
{
    long lo = 0;
    long hi = 0;
    foreach (int weight in weights)
    {
        if (weight > lo)
            lo = weight;
        hi += weight;
    }

    while (lo < hi)
    {
        long mid = lo + ((hi - lo) / 2);
        if (CanShip(weights, maxDays, mid))
            hi = mid;
        else
            lo = mid + 1;
    }
    return checked((int)lo);
}

static bool CanShip(int[] weights, int maxDays, long capacity)
{
    int days = 1;
    long load = 0;
    foreach (int weight in weights)
    {
        if (load + weight > capacity)
        {
            days++;
            load = 0;
            if (days > maxDays)
                return false;
        }
        load += weight;
    }
    return true;
}
```

LowerBound on [1, 2, 2, 4] for target 2 returns 1, the first 2. For target 3 it returns 3, the index of 4, which is the insertion point. For target 9 it returns 4, the length. That last answer is why hi starts at the length.

CanShip assumes every weight fits in the capacity. The search maintains that, because lo starts at the max weight. The sum of weights is a long, so a large array does not wrap hi. checked((int)lo) is a loud failure if a

caller ever asked this method to return a capacity past Int32.MaxValue. In the interview you can return long instead and skip the cast. Say which one you chose.

Complexity

Lower bound is O(log n) comparisons and O(1) extra memory, after the array is sorted. If you sorted it yourself, add the n log n. MinShipCapacity does O(log S) feasibility checks, where S is the sum of the weights, and each check is O(n). Total O(n log S) time and O(1) extra memory. You do not say log n for the answer-range form. The n is inside the predicate.

Interview trap

Mixing while (lo <= hi) with hi = mid can stick forever when lo and hi differ by one, because mid becomes lo and nothing moves. Pick the half-open form and stay with it. (lo + hi) / 2 overflows. Forgetting that lo must be at least the heaviest package makes CanShip spin, because a package larger than the capacity never fits. Off-by-one on the day counter usually comes from resetting the load and also forgetting to count the last open day, or from counting an extra day at the end.

Say this

Half-open range, midpoint is lo plus half the gap, and the predicate is monotone. On a sorted array that finds the first index at or after the target. On an answer range I binary-search the capacity, and the array walk is only the check. I keep the sum in a long.

Drill

1. Write LowerBound from memory. Trace target 2, target 3, and a target past the end on [1, 2, 2, 4].
2. Variant: UpperBound, the first index whose value is strictly greater than the target. Change the comparison to <= and keep the same bounds.
3. Then write MinShipCapacity once, and say the predicate in one sentence before you write CanShip.

Variant, after you have written it

```
public static int UpperBound(int[] a, int target)
{
    int lo = 0;
    int hi = a.Length;
    while (lo < hi)
    {
        int mid = lo + ((hi - lo) / 2);
        if (a[mid] <= target)
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}
```

On [1, 2, 2, 4] with target 2, UpperBound returns 3. LowerBound returned 1. The gap between them is the run of equals. Time is still O(log n).

Part A, Day 6 — Trees and BST

Outcome. You can walk a binary tree in pre-order, in-order, and post-order, compute depth without confusing it with a shortest-path claim, and explain lowest common ancestor for a BST and for a general binary tree. You know a BST is O(height), and height can be n.

Time box. About 3.5 hours. Draw a seven-node tree and write the three visit orders by hand. Code in-order and both depth methods. Code both LCA methods. The variant is validating a BST with an explicit range, using long bounds so Int32.MinValue has a home.

The idea

A binary tree node has a value, a left child, and a right child, either of which may be null. Pre-order visits the node, then the left subtree, then the right. In-order visits left, node, right. On a binary search tree, in-order yields the values in sorted order, which is the check you use when you are unsure the tree is a BST. Post-order visits both subtrees before the node, which is the order you want when the node depends on the children, as depth does.

Depth here means the number of nodes on the longest path from the root to a leaf. An empty tree has depth 0. A single node has depth 1. Ask which convention they want if they say height, because some people count edges and a single node is then 0. The recursive form is one plus the larger child. The same recursion on a skewed tree is one frame per node. A chain of tens of thousands of nodes can throw StackOverflowException. An explicit queue computes the same number level by level and keeps the frames off the call stack. That is the form you switch to when the tree is untrusted.

A BST keeps smaller values on the left and larger values on the right. Search and insert are O(height). A balanced tree gives you log n. A tree built from sorted inserts is a linked list, and the cost is n. You say that before you promise log n.

Lowest common ancestor in a BST: walk from the root. If both targets are smaller, go left. If both are larger, go right. Otherwise this node is the split, and it is the ancestor. This assumes both values are in the tree. If one is missing, the walk can still return a node, and it is the wrong answer. Ask.

Lowest common ancestor in a general binary tree: search both sides. If a node is one of the targets, or is null, return it. If the left search and the right search both return a node, this node is the ancestor. If only one side returns a node, pass that node up. This also assumes both targets are present. If one is missing, the method returns the one it found.

Worked example

```
public sealed class TreeNode
{
    public int Value;
    public TreeNode? Left;
    public TreeNode? Right;
    public TreeNode(int value) => Value = value;
```

```
}

public static void InOrder(TreeNode? node, List<int> output)
{
    if (node is null)
        return;
    InOrder(node.Left, output);
    output.Add(node.Value);
    InOrder(node.Right, output);
}

public static int MaxDepth(TreeNode? node)
{
    if (node is null)
        return 0;
    return 1 + Math.Max(MaxDepth(node.Left), MaxDepth(node.Right));
}

public static int MaxDepthIterative(TreeNode? root)
{
    if (root is null)
        return 0;

    var queue = new Queue<TreeNode>();
    queue.Enqueue(root);
    int depth = 0;
    while (queue.Count > 0)
    {
        int width = queue.Count;
        for (int i = 0; i < width; i++)
        {
            TreeNode node = queue.Dequeue();
            if (node.Left is not null)
                queue.Enqueue(node.Left);
            if (node.Right is not null)
                queue.Enqueue(node.Right);
        }
        depth++;
    }
    return depth;
}

public static TreeNode? LcaBst(TreeNode? node, int a, int b)
{
    while (node is not null)
    {
        if (a < node.Value && b < node.Value)
            node = node.Left;
        else if (a > node.Value && b > node.Value)
            node = node.Right;
        else
            return node;
    }
    return null;
}
```

```
public static TreeNode? Lca(TreeNode? node, TreeNode p, TreeNode q)
{
    if (node is null || ReferenceEquals(node, p) || ReferenceEquals(node, q))
        return node;

    TreeNode? left = Lca(node.Left, p, q);
    TreeNode? right = Lca(node.Right, p, q);
    if (left is not null && right is not null)
        return node;
    return left ?? right;
}
```

Build the tree 4 with left 2 and right 6, and 2 with left 1 and right 3. In-order is 1, 2, 3, 4, 6. Depth is 3. LcaBst of 1 and 3 is the node 2. LcaBst of 1 and 6 is the node 4, because the targets are on different sides. The iterative depth captures the queue count before the loop that drains that level. If you measure Count inside the loop, newly enqueued children stretch the level and the depth is wrong.

Complexity

A full traversal is O(n) time and O(height) call-stack memory, or O(width) queue memory for the level-order form. BST search is O(height). LCA on a BST is O(height) time and O(1) extra memory in the loop above. LCA on a general tree is O(n) time in the form that searches the whole tree, and O(height) stack. You can stop early once both targets are found. The simple form is the one you write unless they ask for the pruning.

Interview trap

Promising log n on a BST you did not balance. Using value equality for LCA when the tree can hold duplicates or when they handed you node identities. Capturing the queue count after you have already enqueued children. Recursing a skewed tree of untrusted size. Forgetting to ask whether both nodes exist. An int range check that cannot represent a bound outside Int32 for the validation variant.

Say this

In-order on a BST is sorted. Depth is one plus the deeper child, and I switch to a queue when the tree can be a chain. BST cost is the height. LCA in a BST walks to the split. LCA in a general tree is the node where the two searches meet, and I ask whether both nodes are present.

Drill

1. Write InOrder, MaxDepthIterative, and LcaBst from memory. Say the visit order of your seven-node drawing before you run anything.
2. Variant: IsBst. Pass a long lower bound and a long upper bound. A node is valid when its value is strictly inside the range. The left child inherits the node's value as its upper bound.
3. Call it as IsBst(root, long.MinValue, long.MaxValue). State what you do if duplicates are allowed. The version below rejects duplicates.

Variant, after you have written it

```
public static bool IsBst(TreeNode? node, long lo, long hi)
{
    if (node is null)
        return true;
    if (node.Value <= lo || node.Value >= hi)
        return false;
    return IsBst(node.Left, lo, node.Value)
        && IsBst(node.Right, node.Value, hi);
}
```

Checking only that left is smaller than the node misses a left grandchild that is larger than the root. The range carries the ancestors. Time O(n). The long bounds are the overflow-shaped pitfall: an int bound cannot sit strictly outside every int.

Part A, Day 7 — Heaps and PriorityQueue

Outcome. You can take the top K frequent numbers with PriorityQueue, and you can explain merging K sorted lists as the same heap with one current head per list. You know the BCL queue is a min-heap.

Time box. About 3.5 hours. Code top K until the size-K rule is automatic. Then code the merge from the sketch without looking. The variant is the Kth largest element, which is top K with a single answer. Leave 20 minutes to say what Dequeue removes.

The idea

PriorityQueue<TElement, TPriority> is a binary heap. With the default comparer it dequeues the smallest priority first. It is not a max-heap, it is not SortedSet, and it allows two elements with the same priority. SortedSet rejects a duplicate priority unless you invent a tie-break, which is why people reach for it and then lose equal frequencies. Dequeue returns the element. TryDequeue can also hand back the priority.

Top K frequent: count in a dictionary, then keep a min-heap of size K ordered by frequency. For each distinct value, enqueue it with its frequency. When the heap grows past K, dequeue. You just threw away the smallest frequency in the heap, so the heap holds the K largest frequencies seen so far. At the end, dequeue the survivors. Their order is smallest frequency first. If the caller wants them sorted, sort the K items, which is cheap because K is small. If K is larger than the number of distinct values, you return all of them.

Equal frequencies have no stable winner. The heap does not remember input order. If the problem requires a tie-break, put a sequence number in the priority. A tuple (frequency, sequence) uses the default tuple comparer, item by item. You decide which sequence should lose when the heap trims, and you say it. If any tied value is acceptable, you leave the priority as the frequency.

Merge K sorted lists uses the same heap. Each list contributes its current head. The priority is the head value, so the smallest head comes out first. You append it, and if that list has a next element you enqueue the next index. You never insert a whole list. The heap size is at most K. A full concatenation followed by a sort is correct and does more work when the lists are already sorted. Say the worse one, then the heap.

Worked example

```
public static int[] TopKFrequent(int[] nums, int k)
{
    if (k <= 0)
        return Array.Empty<int>();

    var counts = new Dictionary<int, int>();
    foreach (int n in nums)
        counts[n] = counts.GetValueOrDefault(n) + 1;

    // Min-heap of at most k values, priority = frequency.
    var heap = new PriorityQueue<int, int>();
    foreach (var (value, freq) in counts)
    {
```

```
        heap.Enqueue(value, freq);
        if (heap.Count > k)
            heap.Dequeue();
    }

    var result = new int[heap.Count];
    for (int i = result.Length - 1; i >= 0; i--)
        result[i] = heap.Dequeue();
    return result;
}

public static int[] MergeSorted(IReadOnlyList<int[]> lists)
{
    var heap = new PriorityQueue<(int list, int index), int>();
    for (int i = 0; i < lists.Count; i++)
    {
        if (lists[i].Length > 0)
            heap.Enqueue((i, 0), lists[i][0]);
    }

    var result = new List<int>();
    while (heap.TryDequeue(out var current, out _))
    {
        result.Add(lists[current.list][current.index]);
        int next = current.index + 1;
        if (next < lists[current.list].Length)
            heap.Enqueue((current.list, next), lists[current.list][next]);
    }
    return result.ToArray();
}
```

Trace top K on [1, 1, 1, 2, 2, 3] with K = 2. Counts are 1 thrice, 2 twice, 3 once. The heap keeps 1 and 2, and 3 is dequeued when the size hits 3. Filling the result from the end puts the larger frequency first. The problem often accepts any order of the K values. Say that.

MergeSorted on [1, 4, 5], [1, 3, 4], and [2, 6] yields 1, 1, 2, 3, 4, 4, 5, 6. Empty lists are skipped, so they do not enqueue a fake head. Two equal heads are both valid. The heap may emit either first.

Complexity

Counting is O(n). The heap never holds more than K + 1 items, so the count phase of top K is O(d log K) for d distinct values, which is O(n log K) at worst. Memory is O(d) for the map plus O(K) for the heap. Merge is O(N log K) time for N total elements and O(K) heap memory, plus O(N) for the output. Sorting the concatenation is O(N log N). You pick the heap when K is smaller than N, which is the point of the question.

Interview trap

Treating PriorityQueue as a max-heap and keeping the wrong K. Using SortedSet and dropping a second item with the same frequency. Forgetting to skip an empty list, then reading index 0. Claiming the merge is O(N log N) after you have already written the heap. Assuming dequeue order is stable across equal priorities.

Say this

PriorityQueue dequeues the smallest priority. For top K I keep a heap of size K and throw away the smallest frequency. Merge K sorted lists is one current head per list in that same heap, N log K, and I skip empty lists. Ties are unordered unless I put a sequence number in the priority.

Drill

1. Write TopKFrequent from memory, including k <= 0 and the case k greater than the number of distinct values.
2. Variant: Kth largest element in an unsorted array. Keep a min-heap of size K of the values themselves. The root is the Kth largest. Do not sort the array unless you are stating the simpler O(n log n) answer first.
3. Say what the root is after the loop, in one sentence.

Variant, after you have written it

```
public static int KthLargest(int[] nums, int k)
{
    var heap = new PriorityQueue<int, int>();
    foreach (int n in nums)
    {
        heap.Enqueue(n, n);
        if (heap.Count > k)
            heap.Dequeue();
    }
    return heap.Peek();
}
```

Peek returns the smallest of the K largest, which is the Kth largest, without removing it. This assumes k is between 1 and nums.Length. Ask before you index off the end. Time O(n log K), extra memory O(K).

Part A, Day 8 — Graphs

Outcome. You can build an adjacency list, run BFS and DFS, and detect a cycle in a directed graph. You can say how the undirected case differs.

Time box. About 3.5 hours. Build the list from an edge array on paper, then in code. Write the directed cycle DFS with three colors and be able to point at a back edge. Write Kahn's algorithm as the second cycle test. The variant is an undirected cycle, which must ignore the edge back to the parent.

The idea

For nodes numbered 0 through n - 1, an adjacency list is an array of lists. A sparse id space uses Dictionary<int, List<int>> and a HashSet for seen nodes. You build the list once. A matrix is the wrong default unless the graph is tiny and dense, because the matrix is n squared memory and the interview graphs are sparse.

BFS from Day 3 gives you distance on an unweighted graph. DFS gives you reachability and the cycle test. For a directed cycle, color each node unseen, on-stack, or done. A neighbor that is still on the stack is a back edge, and that is a cycle. A neighbor that is done is a cross edge or a forward edge into finished work, and it is not a new cycle. You must run the search from every unseen node, or a cycle in a later component survives.

Kahn's algorithm is the same fact as a queue. Compute indegree. Enqueue every node with indegree 0. Pop a node, append it to the order, and decrement its neighbors. A neighbor that hits 0 is enqueued. If the order contains every node, you have a topological order. If it does not, some nodes still have incoming edges inside a cycle. The short order is the cycle report. BFS here is not a shortest path. It is a drain of the graph.

An undirected cycle is a different predicate. The edge back to the parent is how you arrived, and it is not a cycle. A neighbor that is already seen and is not the parent is a cycle. A self-loop is a cycle. Do not paste the directed coloring onto an undirected graph and then treat the parent as a back edge. You will report a cycle on every tree.

Worked example

```
public static bool HasDirectedCycle(int n, List<(int From, int To)> edges)
{
    var adj = new List<int>[n];
    for (int i = 0; i < n; i++)
        adj[i] = new List<int>();
    foreach (var (from, to) in edges)
        adj[from].Add(to);

    const int Unseen = 0;
    const int OnStack = 1;
    const int Done = 2;
    var color = new int[n];
```

```
    for (int i = 0; i < n; i++)
    {
        if (color[i] == Unseen && DfsCycle(i, adj, color))
            return true;
    }
    return false;
}

static bool DfsCycle(int node, List<int>[] adj, int[] color)
{
    const int OnStack = 1;
    const int Done = 2;
    color[node] = OnStack;
    foreach (int next in adj[node])
    {
        if (color[next] == OnStack)
            return true;
        if (color[next] == 0 && DfsCycle(next, adj, color))
            return true;
    }
    color[node] = Done;
    return false;
}

public static List<int> TopologicalOrder(int n, List<(int From, int To)> edges)
{
    var adj = new List<int>[n];
    var indegree = new int[n];
    for (int i = 0; i < n; i++)
        adj[i] = new List<int>();
    foreach (var (from, to) in edges)
    {
        adj[from].Add(to);
        indegree[to]++;
    }

    var queue = new Queue<int>();
    for (int i = 0; i < n; i++)
    {
        if (indegree[i] == 0)
            queue.Enqueue(i);
    }

    var order = new List<int>(n);
    while (queue.Count > 0)
    {
        int node = queue.Dequeue();
        order.Add(node);
        foreach (int next in adj[node])
        {
            indegree[next]--;
            if (indegree[next] == 0)
                queue.Enqueue(next);
        }
    }
```

```
    return order.Count == n ? order : new List<int>();
}
```

Edges (0, 1), (1, 2), (2, 1) are a directed cycle. DFS colors 1 as on-stack, reaches 2, and from 2 sees 1 still on the stack. Kahn enqueues 0, then never drains 1 and 2, so the order is short and the method returns an empty list. Returning empty is a choice. You can return the partial order and let the caller compare the count. Say which one you did. A caller who forgets to check the count will schedule a cyclic graph.

The recursive DFS has the same depth limit as any other recursion. A long chain is a deep stack. If the node count can be large, say you would move the DFS onto an explicit stack and keep the color array. Do not discover that at the end of a 40-minute slot if the constraint says a hundred thousand nodes.

Complexity

Building the list is O(n + e). Directed DFS is O(n + e) time and O(n) memory for the colors, plus O(height) call stack. Kahn is O(n + e) time and O(n) memory for the queue and the indegree array. A missing node id past n - 1 is an input error. Ask whether node ids are dense before you allocate the array.

Interview trap

Reporting a directed cycle when you see any seen node, including a finished one, which flags a diamond as a cycle. Forgetting to start DFS from every component. Treating a short topological order as a successful schedule. Using DFS distance as a shortest path. On an undirected graph, counting the parent edge as a cycle. A dictionary adjacency list with graph[node] when the node has no list.

Say this

I build an adjacency list. BFS for unweighted distance, DFS for reachability. A directed cycle is a back edge to a node still on the stack, or a topological order that does not include everyone. An undirected cycle ignores the edge back to the parent. I start the search from every node.

Drill

1. Write HasDirectedCycle from memory and trace a three-node cycle and a diamond that is not a cycle.
2. Variant: undirected cycle. The adjacency list stores both directions. DFS takes the parent index. Skip the parent. A seen neighbor that is not the parent is a cycle.
3. Say the time cost in n and e, not just n.

Variant, after you have written it

```
public static bool HasUndirectedCycle(int n, List<(int A, int B)> edges)
{
    var adj = new List<int>[n];
    for (int i = 0; i < n; i++)
        adj[i] = new List<int>();
    foreach (var (a, b) in edges)
    {
        adj[a].Add(b);
```

```
        adj[b].Add(a);
    }

    var seen = new bool[n];
    for (int i = 0; i < n; i++)
    {
        if (!seen[i] && DfsUndirected(i, -1, adj, seen))
            return true;
    }
    return false;
}

static bool DfsUndirected(int node, int parent, List<int>[] adj, bool[] seen)
{
    seen[node] = true;
    foreach (int next in adj[node])
    {
        if (next == parent)
            continue;
        if (seen[next] || DfsUndirected(next, node, adj, seen))
            return true;
    }
    return false;
}
```

Parent -1 is the start, and no real node has that id in a 0-based graph. A self-loop has next == node, which is seen and is not the parent, so it returns true. Time O(n + e).

Part A, Day 9 — Recursion and backtracking

Outcome. You can generate subsets by choosing and undoing, and you can explain permutations as the same search with a used set. You copy the path when you record an answer.

Time box. About 3.5 hours. Write subsets until the undo is muscle memory. Trace the call tree for three numbers on paper, including the copy. Then write the permutation variant. Spend the last 20 minutes on when this recursion is safe and when a different recursion is a stack overflow.

The idea

Backtracking is a depth-first search over decisions. For subsets, the decisions are in index order: take this element, or skip it, and never look backward, so you do not emit the same subset in two orders. You record a copy of the current path at each node, including the empty path at the start. Then you add the element, recurse from the next index, and remove the element. The remove is the backtrack. Without it, every later subset inherits the earlier choice.

The path is one list you mutate. If you add that list into the result, every slot in the result points at the same list, and at the end every slot is empty, because you undid every add. The copy is new List<int>(path). That allocation is the output, and it is required.

Permutations use the same undo with a different branch. At each depth you try every unused element, mark it used, recurse, and unmark it. The depth is n, and the branching is n, then n - 1, then n - 2. You record a copy only when the path length is n. Subsets record at every node. Do not mix those two lines.

The call depth of subsets or permutations is n, and n is small because the output is already 2^n or n!. The recursion that overflows in C# is the one that follows a chain: a skewed tree, a linked list, a graph path of tens of thousands. Tail calls are not a feature you can rely on. If a backtracking constraint says n can be 100, the issue is time, not the stack. If a different problem says the list can be 100,000 long, you stop recursing.

Worked example

```
public static List<List<int>> Subsets(int[] nums)
{
    var result = new List<List<int>>();
    var path = new List<int>();
    Walk(0);
    return result;

    void Walk(int start)
    {
        result.Add(new List<int>(path));
        for (int i = start; i < nums.Length; i++)
        {
            path.Add(nums[i]);
            Walk(i + 1);
            path.RemoveAt(path.Count - 1);
        }
```

```
    }
}
```

Trace [1, 2]. The first record is the empty list. Take 1, record [1]. Take 2, record [1, 2]. Undo 2, undo 1. Take 2 from the start index, record [2]. Undo 2. The result has four lists, and they are four different objects. Start at i + 1 so 2 is never followed by 1. That is what keeps [1, 2] and [2, 1] from both appearing. If the problem wants permutations, that restriction is wrong. If the input has duplicates and the problem wants unique subsets, sort first and skip a duplicate element at the same depth. Ask before you write that skip.

Complexity

There are 2^n subsets. Recording each costs time proportional to its length, so the total time is O(n 2^n). Memory is the output plus O(n) for the path and the call stack. Permutations are O(n * n!) time for the same reason: n! leaves, each copied at size n. You say the output size out loud so an exponential answer sounds like a choice and not like a mistake. If they need one subset that matches a constraint, you stop at the first match and the worst case is still exponential.

Interview trap

Adding the path list by reference, so every result mutates together. Forgetting the undo. Starting the next loop at i instead of i + 1 and either looping forever or emitting duplicates, depending on the rest of the code. Recursing on start and also on start + 1 without a structure, and emitting each subset many times. Treating this depth-n search as unsafe, and treating a recursive walk of a 100,000-node chain as safe. The second one is the stack overflow.

Say this

I choose or skip in index order, I copy the path when I record it, and I undo the choice on the way back. Subsets are n times 2 to the n. Permutations are the same undo with a used set, and I copy only at depth n. I recurse here because n is small. I do not recurse down a chain that can be huge.

Drill

1. Write Subsets from memory. Trace [1, 2] and confirm four distinct lists.
2. Variant: permutations of distinct integers. Use a bool array for used, or a HashSet if you prefer. Copy the path only when its count equals n.
3. Say the complexity before you run it, and run it on three elements so you expect six results.

Variant, after you have written it

```
public static List<List<int>> Permutations(int[] nums)
{
    var result = new List<List<int>>();
    var path = new List<int>(nums.Length);
    var used = new bool[nums.Length];
    Walk();
    return result;
```

```
    void Walk()
    {
        if (path.Count == nums.Length)
        {
            result.Add(new List<int>(path));
            return;
        }

        for (int i = 0; i < nums.Length; i++)
        {
            if (used[i])
                continue;
            used[i] = true;
            path.Add(nums[i]);
            Walk();
            path.RemoveAt(path.Count - 1);
            used[i] = false;
        }
    }
}
```

The bool array is the used set. A HashSet<int> of indices is the same idea and allocates more. Time O(n * n!), stack depth n.

Part A, Day 10 — Dynamic programming starter, and how to talk through a question

Outcome. You can write climbing stairs with two running totals, and coin change as a one-dimensional table, including the case with no solution. You can talk through a coding question in a fixed order: clarify, brute force, complexity, code, test. You can say why greedy fails on coin change.

Time box. About 3.5 hours. Spend 40 minutes on the two DP methods and the greedy counterexample. Spend 40 minutes rewriting them from memory. Spend 30 minutes on the house-robber variant. Spend the remaining time on the talk-through, out loud, using two-sum as the script, then once more using coin change. The talk-through is the page you are allowed to use in every later interview, including the ones that are not DP.

The idea

A starter dynamic program is a brute-force recursion you can see repeating itself, rewritten so each smaller input is solved once. If you can define dp[i] as the answer for a smaller prefix or a smaller amount, and you can say the transition in one sentence, you have the starter. Two dimensions can wait. You begin with the recursion even if you never ship it, because the table is that recursion with the calls removed.

Climbing stairs: you may take 1 or 2 steps, and you want the number of ways to reach n. To reach i you came from i - 1 or from i - 2, so ways(i) = ways(i - 1) + ways(i - 2). That is Fibonacci, with ways(1) = 1 and ways(2) = 2. The recursive tree recomputes the same i many times and is exponential, and a deep n is also a stack overflow. You keep two running totals and walk up to n. Time O(n), extra memory O(1).

The totals leave Int32 in the mid-40s of the Fibonacci sequence. If n can be that large, the total is a long. The usual interview n is small. Use a long anyway if you do not want to think about the boundary while you are writing the loop.

Coin change: the fewest coins that sum to the amount, unlimited supply of each coin. dp[a] is the fewest coins that make amount a. dp[0] is 0. Every other cell starts as a sentinel meaning impossible. For each amount, you try each coin that fits, and you take one plus the answer for the remainder. If a cell is still the sentinel at the end, return -1. A coin of 0 or less does not belong in the loop. Greedy, taking the largest coin first, is wrong for some sets. Coins 1, 3, and 4, amount 6: greedy takes 4 + 1 + 1, three coins. The table takes 3 + 3, two coins. You mention that so they do not hear you say DP when greedy would do, and you do not hear yourself say greedy on this problem.

Worked example

```
public static long ClimbStairs(int n)
{
    if (n <= 2)
        return n;

    long prev2 = 1;
```

```
    long prev1 = 2;
    for (int i = 3; i <= n; i++)
    {
        long current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}

public static int CoinChange(int[] coins, int amount)
{
    if (amount < 0)
        return -1;
    if (amount == 0)
        return 0;

    var dp = new int[amount + 1];
    Array.Fill(dp, amount + 1);
    dp[0] = 0;

    for (int a = 1; a <= amount; a++)
    {
        foreach (int coin in coins)
        {
            if (coin <= 0)
                throw new ArgumentOutOfRangeException(nameof(coins));
            if (coin <= a)
                dp[a] = Math.Min(dp[a], dp[a - coin] + 1);
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

ClimbStairs(4) is 5: 1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2. CoinChange of coins [1, 3, 4] and amount 6 is 2. CoinChange of [2] and amount 3 is -1, because 3 stays at the sentinel. The sentinel amount + 1 is larger than any real answer, since you cannot need more coins than the amount when a coin of 1 exists, and when it does not exist the sentinel simply survives. dp[a - coin] + 1 can be the sentinel plus one, which is still larger than any real answer and loses the Math.Min. That is fine.

Throwing on a non-positive coin is a guard, not the algorithm. If you would rather return -1, say so. An infinite loop from a zero coin is the bug the guard prevents.

Complexity

Climbing stairs is O(n) time and O(1) extra memory. The naive recursion is exponential time and O(n) stack. Coin change is O(amount * coins) time and O(amount) memory. You can talk about the recursion with a cache as O(amount * coins) too. The table is that cache, written in order so each remainder is already filled. This order works because the recurrence only looks at smaller amounts.

Interview trap

Writing the recursive Fibonacci and calling it dynamic programming. Using greedy on coin change. An int total that wraps. A zero coin. Forgetting the impossible case and returning the sentinel as if it were a real count. Building a two-dimensional table when one dimension of amount is the whole state.

Say this

Climbing stairs is Fibonacci. I keep two running totals in a long, because the recursive tree repeats work and the int total overflows. Coin change is a table of the fewest coins for each smaller amount, and I return -1 when the amount stays unreachable. Greedy is wrong for coins 1, 3, and 4 with amount 6.

Drill

1. Write both methods from memory. Compute climbing stairs for n = 4 by hand and coin change for [1, 3, 4] and 6 by hand.
2. Variant: house robber. A row of house values, you cannot rob two adjacent houses. dp at each house is the max of skipping it or taking it plus the answer two houses back. Keep two running totals, the same shape as climbing stairs.
3. Say the transition before you write the loop.

Variant, after you have written it

```
public static int Rob(int[] houses)
{
    int skipPrev = 0;
    int takePrev = 0;
    foreach (int money in houses)
    {
        int skip = Math.Max(skipPrev, takePrev);
        int take = skipPrev + money;
        skipPrev = skip;
        takePrev = take;
    }
    return Math.Max(skipPrev, takePrev);
}
```

skip is the best answer for the houses before this one. take is this house plus the best answer that did not include the previous house. Time O(n), extra memory O(1). An int is enough while the sum fits. If the problem says the values can be large, use a long. The adjacent constraint is why this is not a plain sum of the positive houses.

How I talk through a coding question

This is the other half of Day 10, and it is the script for every coding screen, not only for dynamic programming. You talk in this order even when you already see the answer. The interviewer is scoring the order.

1. Clarify. Restate the input, the output, and the question in one sentence. Ask about empty input, duplicates, negative numbers, overflow, and whether you need the original index. Ask for a bound on n if they have one. Agree on a small example, including the answer.
2. Brute force. Say the obvious loops and their cost. Do not apologize. A correct n-squared answer that you can finish is worth more than a silent search for the optimal one. Ask whether n-squared is acceptable before you leave it.
3. Better structure. Name the pattern and why the brute force was doing repeated work: a map for a complement, two pointers for a sorted pair, a queue for a shortest unweighted path, a table for a recurrence that repeats.
4. Complexity. Time and extra memory, including the output when the output is the memory. Say expected when a hash map is involved. Say the height when a tree is involved.
5. Code. Name the variables for what they mean. Handle the empty input first. Leave the C# pitfall in the code, not in a comment you forget: TryGetValue, a long sum, a half-open bound, a copied list.
6. Test. Walk the example. Then the empty input, a single element, a duplicate, and the case that fails if the pitfall is present. If a test fails, say what the invariant was and fix that, rather than nudging a bound until the sample passes.

The script, spoken on two-sum

```
Clarify.
  I return two indices that add to the target, or an empty array.
  I am assuming one answer exists in the sample, the same element
  cannot be used twice, and the numbers fit in an int.
  Example: [2, 7, 11, 15], target 9, indices 0 and 1.

Brute force.
  Every pair, n squared time, constant extra memory.
  I can write that if n is a few thousand. I am assuming n can be larger.

Better.
  One pass. Map from value to index. For each value, look up target - value.
  Store the index after the lookup so a value is not paired with itself.

Complexity.
  Expected linear time, linear memory.

Code.
  Dictionary, TryGetValue, return the earlier index then this one.

Test.
  The sample.
  [3, 3], target 6, indices 0 and 1.
  A single element, empty result.
  If they allow values near Int32 boundaries, the complement moves to a long.
```

Then you stop talking and type. If you get stuck, you say which step you are on. Stuck in the code is not a reason to abandon the tests. Stuck in the optimal structure is a reason to write the brute force and then improve one loop.

The same script, one paragraph, on coin change

Clarify: fewest coins to make the amount, unlimited counts, -1 if it is impossible, coins are positive. Brute force: try every coin and recurse on the remainder, which repeats the same remainder. Better: dp[a] is the fewest coins for a, filled from 0 to amount. Complexity: amount times the number of coins, and a table of size amount + 1. Test: amount 0 returns 0, coins [1, 3, 4] and amount 6 returns 2, coins [2] and amount 3 returns -1. Greedy is the wrong improvement. Then code.

Say this

I clarify the empty case and the overflow, I say the brute force and its cost, I name the structure that removes the repeated work, and I test the sample plus the case that breaks. If I cannot see the optimal answer yet, I write the brute force and improve it. I do not go quiet.

What you can skip

You can skip segment trees, heavy-light decomposition, and contest tricks that exist to shave a log factor off an already acceptable solution. You cannot skip the ten shapes above. A tech-lead screen uses them as a way to watch you code in C# under a clock: collections, bounds, and whether you test. The system-design half of this guide starts next. It assumes you can still say, in one minute, how you would attack a small coding question if they drop one into the whiteboard.

Part B — System design for the whiteboard

Ten days. Day 1 is the checklist you write on the board before any product name. Day 2 teaches the order API in full, and later days reuse it. The load numbers are assumptions: about 50 requests a second at peak, about 2 million orders a year, a read p95 under 300 milliseconds, email within a minute. Label them. If the panel changes a number, change the design. A million events a second is a different system, and you do not design it unless they ask.

The stack stays ASP.NET Core, EF Core, SQL Server or Azure SQL, Azure App Service for the request API, Azure Functions on the isolated worker for event and schedule work, Azure Service Bus when a consumer must finish under a lock, Entra ID for the caller, and managed identity for the process. A second deployable appears only when you can name its data and its failure. The microservices chapter in the roadmap and the deep dive is where that cut is practiced. Here you point at it and move on.

Part B, Day 1 — The design spine

Outcome. In three minutes you can put a checklist on the board that survives an interruption, and in twenty minutes you can walk it on a .NET backend without jumping to a platform you do not operate.

Time box. About 3.5 hours. Read the checklist once. Then close the page and redraw it twice from memory. Spend an hour applying it to a returns endpoint, on paper, with no new Azure product. Spend the last hour giving the twenty-minute version out loud, and stop when the timer ends even if you love the next box.

The prompt

The panel says: design the backend. The resource is vague on purpose. Your job is to make the problem specific enough to draw, small enough to finish, and honest about what you refused. You are the tech lead. You still name the API, the table, the transaction, and the rollback.

Assumptions

If they give no numbers, use the order-system assumptions and invite a correction: about 50 requests a second at peak, about 2 million orders a year, read p95 under 300 milliseconds, email within a minute, one region to start. Out of scope until they pull it in: a million-event stream, a second team, a synchronous payment capture, multi-region active-active. Write the out-of-scope line on the board so a later interruption has somewhere to land.

API

Name the resource, the verb, and the status code before the framework. POST creates, and a partner retry is part of the contract, so the create takes an idempotency key. GET by id is the read. A list uses a cursor, not a page offset, once the list can grow. PATCH carries a rowversion when two people can edit. Status codes you must be able to place without a pause: 201 the first create, 200 a replay of the same request, 400 a body you will not accept, 401 the caller is unknown, 403 the caller is known and not allowed, 404 the id is not visible in

this tenant, 409 a lost update or a key reused with a different body, 429 a limiter you chose to enforce, 503 a required dependency you gave up on. 500 is an unexpected fault with a correlation id, not a validation failure.

Data

Name the aggregate, the primary key, the business key, the tenant column, and the index that matches the hot query. The hot order list is (TenantId, CreatedUtc, Id), with the columns the list returns included so the query does not go back to the row for those fields. A uniqueness rule that is true in the domain becomes a unique index, and the tenant is part of that index. The application login can insert, update, and select. It cannot change the schema.

Consistency

Draw one box around what commits together. For the order that box is the order, its lines, the idempotency row, and the outbox row. Draw a second box around what is allowed to lag: the email, the search copy, the cache. Say the lag in the same units as the assumption, here within a minute for mail. A distributed transaction across SQL and Service Bus is the thing you refuse. The outbox is how the message still happens after the commit.

Failure

Walk five failures, each with what the user sees. The database is down: the API returns 503, and the health check fails so the instance leaves rotation. The bus is down after the commit: the order stays, the outbox retries. The partner times out and posts again with the same key: they get the original order. Two workers update the order: one gets 409 and re-reads. A message cannot be processed: it dead-letters, the order is not deleted, someone is paged on the dead-letter depth. You do not need a sixth failure in the first twenty minutes.

Operations

Three identities, every time. The caller's Entra token decides what the human or the partner app may do. The API's managed identity is the SQL user app_orders_runtime, limited to the schema it writes. The pipeline's migration identity applies DDL and is not the login the site uses all day. A function that sends mail can be a fourth identity if its rights are smaller. Logs carry a correlation id, the tenant, and the order id, and they do not carry a full address or a secret. The dashboard you name is request latency, SQL duration, outbox age, and dead-letter depth. Deploy is a reviewed migration, then a slot, then a smoke test, then a swap. Rollback is the swap back, and only while the schema is still one the old bits can run. That means expand, then contract later.

The tradeoff refused

You refuse four things in the first slice, and you say them so the panel knows you saw them. You refuse a microservice per noun. You refuse a shared SQL login in a high-privilege role. You refuse to publish to Service Bus inside the SQL transaction. You refuse to call Database.Migrate on startup, because a bad migration then

takes every instance down during boot, and two instances can race. The pipeline applies the script before traffic moves.

Sketch

The board is the sketch. The pipeline order is the part you can also write as code, and it matches the request path you study in the deep dive and in the pipeline guide. Exception handling wraps the rest. Authentication runs before authorization. The rate limiter runs before the endpoint when the endpoint is what you are protecting. Correlation id is early so a 500 and a SQL log share it.

```
Board, first three minutes:

  User and success: partner creates an order, customer reads it, mail can lag.
  Assume: 50 req/s peak, 2 million orders/year, read p95 < 300 ms, mail < 1 min.
  Out of scope: multi-region, a pricing service, a million-event stream.

  API:    POST /orders, GET /orders/{id}, GET /orders?cursor=, PATCH with rowversion
  Data:   orders schema, TenantId, unique (TenantId, IdempotencyKey), rowversion
  Commit: order + lines + idempotency + outbox
  After:  relay publishes, function sends mail, consumer dedupes
  Fail:   503, retry, 409, dead-letter
  Ops:    Entra token, app_orders_runtime, pipeline migrator, slot swap
  Refuse: extra service, db_owner, two-phase commit, Migrate at startup

// The request path you are willing to write if they ask.
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers();
app.MapHealthChecks("/health/ready");
```

Health for readiness includes a short SQL check. Liveness does not, or a SQL blip restarts every instance. Always On stays on for the API so the process is up. ARR affinity stays off. Those are operating choices from the Azure day of the deep dive, and you repeat them here because a second instance is the first interruption you will get.

What to say in 20 minutes

1. Minute 0 to 2. Read the assumptions and the out-of-scope line. Ask them to correct the number.
2. Minute 2 to 6. API and the status codes, including the retry.
3. Minute 6 to 10. Tables, the unique key, the index for the list, the SQL role.
4. Minute 10 to 14. The one transaction, then the outbox, then the lag.
5. Minute 14 to 17. The five failures and what the caller sees.
6. Minute 17 to 20. Identities, the swap, and the tradeoff you refused. Stop.

Practice this clock on a returns API the same evening: POST /returns is a new aggregate, it references an order, it has its own idempotency key, and it does not become a new service. If you cannot finish returns in twenty minutes, the spine is not yours yet.

Interview trap

Opening with a cluster, a cache, and six services before you have named the user. Inventing a quota or a price when you do not remember the document. Mixing 401 and 403. Putting the page offset on a list that will grow. Hiding the SQL role. Spending the last five minutes on a product you would not operate next month.

Say this

I write the user, the assumed number, and the out-of-scope line first. Then the API, the transaction, the failure the caller sees, and the rollback. I keep one API and one database at this load. A second process has to bring its own data, and that conversation lives in the microservices chapter, not in the first three minutes.

Part B, Day 2 — The order API

Outcome. You can teach the order API from the header to the dead-letter: idempotency, rowversion, outbox, App Service, and a Function on the isolated worker. This is the anchor. Later days only add a workload. The short version sits in the deep dive. This day is the one you can redraw and code.

Time box. About 3.5 hours. Spend 40 minutes reading the flow once. Spend an hour writing the SQL and the create path from memory. Spend 40 minutes on the relay lease and the consumer. Spend the rest of the time giving the twenty-minute version twice, with a friend or a timer interrupting on the second instance.

The prompt

Design the backend for creating an order, reading it, and emailing a confirmation. Partners retry when they time out. Two staff members must not lose each other's updates. You are the tech lead. Ship one slice a team can operate.

Assumptions

About 50 requests a second at peak, about 2 million orders a year, read p95 under 300 milliseconds, email within a minute. One region. The partner is an Entra application using an app role. A human user, if you have one, uses a delegated scope. The server prices the order. The client sends a sku, a quantity, and a ship-to. The client does not send the total, a tenant id, or an admin flag. Pricing is a module behind an interface in this process, not a service. Say that the numbers are assumptions.

API

Call	Contract	Code
POST /orders	Header Idempotency-Key. Body is sku, quantity, ship-to. Raw body bytes are the hash.	201 the first time, with Location. 200 when the same key and the same bytes are retried. 409 when the key is reused with different bytes. 400 when the key or the body is missing or invalid.
GET /orders/{id}	Projection, no tracking. The id must be in the caller's tenant.	200 or 404. 404 when the row exists for another tenant, so you do not reveal it.
GET /orders?cursor=&limit=	Keyset on (CreatedUtc, Id), limit capped. The cursor is the last row's created time and id.	200 and the next cursor. An offset is the design you mention and do not ship.
PATCH /orders/{id}	The client sends the rowversion it read. Status moves only through a rule you can say.	200, 404, or 409 when the version lost or the transition is illegal.
Auth failures	No token, bad token, or missing role.	401 unknown. 403 known and not allowed.

You require the idempotency key on create. A create without a key is a duplicate waiting for a timeout. That requirement is a product choice, and you label it. GET does not need a key. The first response is 201. The replay is 200 with the stored order id, so the client can tell a retry from a first insert if it cares, and either way it has the id.

Data

```
CREATE TABLE orders.Orders
(
    Id           uniqueidentifier NOT NULL CONSTRAINT PK_Orders PRIMARY KEY,
    TenantId     uniqueidentifier NOT NULL,
    Number       nvarchar(40)     NOT NULL,
    Status       nvarchar(32)     NOT NULL,
    Total        decimal(18, 2)   NOT NULL,
    RowVersion   rowversion       NOT NULL,
    CreatedUtc   datetime2        NOT NULL
);

CREATE UNIQUE INDEX UX_Orders_Tenant_Number
    ON orders.Orders (TenantId, Number);

CREATE INDEX IX_Orders_Tenant_Created
    ON orders.Orders (TenantId, CreatedUtc DESC, Id DESC)
    INCLUDE (Status, Total);

CREATE TABLE orders.IdempotencyKeys
(
    TenantId       uniqueidentifier NOT NULL,
    KeyValue       nvarchar(128)    NOT NULL,
    RequestHash    binary(32)       NOT NULL,
    OrderId        uniqueidentifier NOT NULL,
    ResponseStatus int              NOT NULL,
    CreatedUtc     datetime2        NOT NULL,
    CONSTRAINT PK_IdempotencyKeys PRIMARY KEY (TenantId, KeyValue)
);

CREATE TABLE orders.Outbox
(
    Id              uniqueidentifier NOT NULL CONSTRAINT PK_Outbox PRIMARY KEY,
    Type            nvarchar(64)     NOT NULL,
    Payload         nvarchar(max)    NOT NULL,
    CreatedUtc      datetime2        NOT NULL,
    PublishedUtc    datetime2        NULL,
    LeasedUntilUtc  datetime2        NULL,
    LeaseOwner      nvarchar(64)     NULL
);

CREATE INDEX IX_Outbox_Unpublished
    ON orders.Outbox (CreatedUtc)
    WHERE PublishedUtc IS NULL;
```

The hash is SHA-256 of the raw request body, not of a DTO you serialized again. Two JSON texts that differ only by whitespace are different bodies under that rule, and the client must retry the same bytes. That is the contract you document. Re-hashing a reordered object is how a retry becomes a false 409. Rowversion is not the idempotency mechanism. The key is for a retry of the same request. The rowversion is for two different updates of an order that already exists.

Consistency

The create commits the order, the key, and the outbox row in one SQL transaction. If that commit succeeds, the order is durable and a relay will see the outbox row. If it fails, none of the three exist, and the client can retry. You do not insert the key in a transaction of its own and the order in another. A crash in between is a key that points at nothing, or an order with no key.

The publish sits outside the transaction. A relay leases unpublished rows, sends them to Service Bus with MessageId set to the outbox id, then sets PublishedUtc. A crash after the send and before the mark sends the message again after the lease expires. The consumer treats that as normal. Service Bus duplicate detection, if you turn it on, has a history window you configure. You do not memorize the window, and you do not depend on it alone, because a duplicate after the window is still a duplicate to the business. The consumer's unique key is the one you can explain at night.

The API does not wait for the mail provider. 201 means the order and the outbox row committed. Mail within a minute is the assumption, measured from commit to the provider accepting the message, and you watch outbox age to see if you are inside it. Two concurrent creates with the same key: one commits, the other hits unique index 2601 or unique constraint 2627, rolls back, clears the change tracker, and reads the winner. Same hash returns 200. Different hash returns 409.

Failure

What broke	What you do	What the caller sees
SQL is down	Readiness fails. The instance leaves the load balancer. You do not retry the business create in a loop inside the request.	503
The partner times out after the commit	They retry the same bytes and the same key.	200 and the original order id
The partner retries with a different body	The stored hash does not match.	409
Service Bus is down	The outbox row stays unpublished. The relay tries again. The order is already saved.	201 already happened
Two staff PATCH the same order	The WHERE includes the rowversion. Zero rows is a concurrency exception.	409, then the client re-reads
The mail payload can never succeed	The subscription dead-letters it. The order stays placed.	The customer still has an order. Support sees the dead letter.

Operations

The API runs on Azure App Service. Always On is on. ARR affinity is off. A staging slot gets the new bits, a smoke test hits it, and you swap. The Function app runs the isolated worker. One timer function relays the outbox. One Service Bus function sends mail. Each process uses a managed identity. app_orders_runtime may insert and update the orders schema and may not alter it. The relay's identity may read and update the outbox and send to the topic. The mail function's identity may write a delivery row and may not delete orders. The pipeline identity applies migrations and is not in the application connection string. Connection strings use Authentication=Active Directory Managed Identity. There is no SQL password in app settings.

Rollback is a slot swap back, and you say the condition: this release only expanded the schema, so the previous bits still run. A dropped column means you cannot swap back, so you do not drop a column in the same release as the code that stops writing it. Messages already published are not recalled. Consumers accept the current payload and the previous one for that release. A feature flag can stop the relay if the consumer is bad. You do not delete orders to undo a mail bug.

A second API instance is the normal case. Idempotency lives in the unique key, not in a static dictionary. The rate limit, when you add Day 7, is per instance unless a gateway holds the shared budget. The relay lease uses UPDLOCK and READPAST so two relay instances skip locked rows. If both still publish, the MessageId keeps the consumer idempotent. You do not run Database.Migrate from either instance.

The tradeoff refused

You refuse a two-phase commit between SQL and Service Bus. You refuse to send mail inside the request. You refuse to accept the total from the client. You refuse a pricing microservice in this slice. You refuse an in-memory idempotency cache. You refuse receive-and-delete on the subscription, because a crash after receive and before send loses the only copy of the work. Peek-lock is the mode, and complete happens after the work is durable.

Sketch

```
[Authorize(Policy = "OrdersWrite")]
[HttpPost("/orders")]
public async Task<IActionResult> Create(CancellationToken ct)
{
    if (!Request.Headers.TryGetValue("Idempotency-Key", out var keyHeader)
        || string.IsNullOrWhiteSpace(keyHeader))
        return Problem(statusCode: 400, title: "Idempotency-Key is required.");

    using var reader = new StreamReader(Request.Body);
    string raw = await reader.ReadToEndAsync(ct);
    var body = JsonSerializer.Deserialize<CreateOrderRequest>(raw);
    if (body is null || body.Quantity <= 0)
        return Problem(statusCode: 400, title: "The order body is invalid.");

    byte[] hash = SHA256.HashData(Encoding.UTF8.GetBytes(raw));
    Guid tenantId = _tenant.TenantId;
    decimal total = await _pricing.PriceAsync(body.Sku, body.Quantity, ct);
    var order = new Order
```

```
    {
        Id = Guid.NewGuid(),
        TenantId = tenantId,
        Number = AllocateNumber(),
        Status = "Placed",
        Total = total,
        CreatedUtc = DateTime.UtcNow
    };

    await using var tx = await _db.Database.BeginTransactionAsync(ct);
    try
    {
        _db.Orders.Add(order);
        _db.IdempotencyKeys.Add(new IdempotencyKey
        {
            TenantId = tenantId,
            KeyValue = keyHeader.ToString(),
            RequestHash = hash,
            OrderId = order.Id,
            ResponseStatus = StatusCodes.Status201Created,
            CreatedUtc = DateTime.UtcNow
        });
        _db.Outbox.Add(new OutboxMessage
        {
            Id = Guid.NewGuid(),
            Type = "OrderPlaced",
            Payload = JsonSerializer.Serialize(new { order.Id, order.TenantId }),
            CreatedUtc = DateTime.UtcNow
        });
        await _db.SaveChangesAsync(ct);
        await tx.CommitAsync(ct);
    }
    catch (DbUpdateException ex) when (IsUniqueViolation(ex))
    {
        await tx.RollbackAsync(ct);
        _db.ChangeTracker.Clear();
        return await ReplayOrConflict(tenantId, keyHeader.ToString(), hash, ct);
    }

    return Created($"/orders/{order.Id}", new { orderId = order.Id });
}

static bool IsUniqueViolation(DbUpdateException ex) =>
    ex.InnerException is SqlException sql && sql.Number is 2601 or 2627;
```

ReplayOrConflict loads the key on a clean tracker. A matching hash returns 200 and the stored order id. A different hash returns 409. If the key row is absent, the unique violation was some other index, such as the order number, and that is a 409 with a different title. ChangeTracker.Clear matters: after the failed SaveChanges the context still holds the inserts, and the next SaveChanges would try them again.

```
public async Task<IActionResult> Patch(Guid id, UpdateOrder body, CancellationToken ct)
{
    var order = await _db.Orders.FirstOrDefaultAsync(o => o.Id == id, ct);
    if (order is null)
```

```
        return NotFound();

    _db.Entry(order).Property(o => o.RowVersion).OriginalValue =
        Convert.FromBase64String(body.RowVersion);
    order.Status = body.Status;
    try
    {
        await _db.SaveChangesAsync(ct);
    }
    catch (DbUpdateConcurrencyException)
    {
        return Conflict();
    }
    return Ok();
}

-- Two relay instances. Locked rows are skipped. A dead worker's lease expires.
;WITH due AS
(
    SELECT TOP (50) *
    FROM orders.Outbox WITH (ROWLOCK, READPAST, UPDLOCK)
    WHERE PublishedUtc IS NULL
      AND (LeasedUntilUtc IS NULL OR LeasedUntilUtc < SYSUTCDATETIME())
    ORDER BY CreatedUtc
)
UPDATE due
SET LeasedUntilUtc = DATEADD(minute, 2, SYSUTCDATETIME()),
    LeaseOwner = @worker
OUTPUT inserted.Id, inserted.Type, inserted.Payload;
```

The two-minute lease is a choice you make so a crashed worker does not block the row forever. It is not a platform default. After a successful send, set PublishedUtc. Do not set it before the send. The Service Bus message uses the outbox id as MessageId. The timer that calls the lease is a Function on the isolated worker, a few times a minute, so the outbox age stays inside the one-minute mail assumption. The exact cron is an operating choice. The function timeout for the plan you are on is a number you look up, and one batch of 50 small publishes fits in a wide margin. You do not put the mail provider call in this relay if Day 6 is going to fan out. The relay only publishes.

```
// App Service / Function application settings. Names, not secrets.
// ServiceBus__fullyQualifiedNamespace: orders.servicebus.windows.net
// ConnectionStrings__Orders:
//   Server=tcp:orders.database.windows.net,1433;Database=orders;
//   Authentication=Active Directory Managed Identity;Encrypt=True;
```

What to say in 20 minutes

1. Minute 0 to 2. Partner retries, server prices, the four assumptions, one region.
2. Minute 2 to 6. POST with the key, 201 versus 200 versus 409, GET, PATCH with the rowversion. The client cannot send the total.

3. Minute 6 to 11. The three tables, the unique key, the filtered outbox index, the list index. Hash the raw bytes.
4. Minute 11 to 15. One transaction. Relay after commit. MessageId is the outbox id. Consumer dedupes. Lease so two relays skip locked rows.
5. Minute 15 to 18. 503, replay, 409, dead-letter. The order survives a mail failure.
6. Minute 18 to 20. App Service, affinity off, Function isolated worker, three identities, swap back only on an expanded schema. Pricing stays a module.

Interview trap

Using the rowversion as the retry key. Hashing a re-serialized DTO. Committing the outbox in a different transaction from the order. Completing the message before the send, or using receive-and-delete. Calling Migrate on startup. Drawing a pricing service before the idempotency row exists. Forgetting ChangeTracker.Clear after the unique violation. Caching the idempotency key in memory and missing the other instance.

Say this

One transaction holds the order, the idempotency row, and the outbox. The relay publishes after commit, the consumer dedupes on the message id, and a lost update returns 409 from the rowversion. The API is App Service, the worker is a Function on the isolated worker, and the SQL login cannot migrate. I hash the raw body. I do not accept the price from the client.

Part B, Day 3 — Catalog reads and order writes

Outcome. You can put a read-heavy catalog next to the write-heavy order path, choose a cache only where a staleness budget exists, page with a keyset, and say when you still do not split a service.

Time box. About 3.5 hours. Redraw the order transaction from Day 2 in ten minutes so you do not design the catalog by forgetting it. Then the catalog read path, the keyset, and the cache key. Spend the last hour defending, out loud, the decision to keep one deployable.

The prompt

The same company has a product catalog that is read constantly and an order path that is the system of record. Design both. The panel will ask why the catalog is not its own service. Answer that with a condition, not with a slogan.

Assumptions

The order assumptions still hold: about 50 requests a second at peak across the API, read p95 under 300 milliseconds. Catalog point reads are inside that budget. A product description may be up to 60 seconds stale. That 60 seconds is a product choice, not a cache product's limit. The price on the order is current at the moment of purchase, so the price used for an order is read from SQL inside the create, not from the description cache. One team owns both. Search with ranking is out of scope. If they add a real search box with its own relevance needs, you name a search index then, and you still do not start there.

API

GET /products/{id} returns the description the page needs. GET /products?cursor=&limit= is the admin or the browse list, keyset on (Name, Id) or on the product id if that is the sort. POST /orders is unchanged from Day 2 and calls the pricing module on the way into the transaction. The catalog write, when a merchandiser edits a description, is a separate command with its own rowversion. It does not share a transaction with an order. There is no public endpoint that returns another tenant's product. 404 hides a cross-tenant id.

Data

```
CREATE TABLE catalog.Products
(
    Id          uniqueidentifier NOT NULL CONSTRAINT PK_Products PRIMARY KEY,
    TenantId    uniqueidentifier NOT NULL,
    Sku         nvarchar(64)     NOT NULL,
    Name        nvarchar(200)    NOT NULL,
    Description nvarchar(max)    NULL,
    UnitPrice   decimal(18, 2)   NOT NULL,
    UpdatedUtc  datetime2        NOT NULL,
    CONSTRAINT UQ_Products_Tenant_Sku UNIQUE (TenantId, Sku)
);
```

```
CREATE INDEX IX_Products_Tenant_Name
    ON catalog.Products (TenantId, Name, Id)
    INCLUDE (Sku, UnitPrice);

-- Order list, repeated so the contrast is on the board.
-- Hot path is a point read by id plus this list:
--   (TenantId, CreatedUtc DESC, Id DESC) INCLUDE (Status, Total)

SELECT TOP (@limit) Id, Name, Sku
FROM catalog.Products
WHERE TenantId = @tenant
  AND (Name > @afterName OR (Name = @afterName AND Id > @afterId))
ORDER BY Name, Id;
```

OFFSET and FETCH for a deep page makes SQL read and throw away every skipped row. A keyset asks for the next page after the last key you returned. The sort must be unique, so Name alone is not enough and the id breaks the tie. The cursor is opaque to the client in the sense that you do not promise they can edit it, and it is still just the last key, so you can explain it.

Consistency

Orders stay read-your-writes from the primary. You do not put the create behind a cache, and you do not serve the GET-by-id that follows a 201 from a replica that might lag, not at this scale, because you have not measured a need. The description cache is allowed to lag for the 60-second budget. A price change is visible to the next order immediately because pricing reads SQL. If the business cannot accept a stale description either, you drop the cache and keep the keyset. At 50 requests a second a primary-key lookup is a reasonable thing to ask Azure SQL to do. You add a cache because a page got expensive or because you measured CPU, not because a diagram had a cylinder missing.

IMemoryCache is per instance. An edit on instance A does not clear instance B. That is acceptable for a description inside a labeled budget, and it is unacceptable for a price you promised was current. A distributed cache gives you a shared key and a shared delete, and it adds a dependency. You take that dependency when the invalidation has to be prompt across instances, or when the cached object is expensive to rebuild and you have already measured the stampede. You do not take it on day one for a point read.

Failure

SQL down means both the order create and the catalog read fail ready, and the API returns 503. A cache that still holds descriptions can serve those keys, and you may do that only if you are explicit that the page is cached and the price is not on it. A poisoned cache key, for example a key that omitted the tenant, leaks one tenant's description to another. That failure is worse than a slow page. The key includes the tenant id and the product id and the fact that it is the description, not the price. Output caching of a whole response has the same bug if the vary key forgets the tenant. For an authenticated tenant API you cache inside the service, where the key is obvious, and you leave response caching for a page that is truly identical for every caller.

Operations

One App Service, one Azure SQL database, two schemas, two SQL users if you want the catalog writer and the order writer separated. The API identity can read products and write orders. A merchandiser role in Entra is allowed to PATCH a product. The list endpoint's index is part of the PR, with the filter and a sample plan, which is the same review rule as the rest of the roadmap. You watch SQL duration and cache hit rate if a cache exists. A hit rate with no latency win is a cache you delete.

The tradeoff refused

You refuse to split the catalog into its own service while one team owns it, the order transaction only needs a price lookup, and you have not measured a read workload that hurts the order path. A split is justified later by a release train, a measured resource fight, or a team that will carry the pager. The deep dive's microservices day has the cut: own schema, a contract test, and an owner. You can point at that day. You do not redraw it here. You also refuse a read replica at 50 requests a second, a cache of the order command, and OFFSET paging on the hot list.

Sketch

```
public async Task<ProductDescription?> GetDescription(Guid id, CancellationToken ct)
{
    string key = $"desc:{_tenant.TenantId:N}:{id:N}";
    if (_cache.TryGetValue(key, out ProductDescription? cached))
        return cached;

    var row = await _db.Products.AsNoTracking()
        .Where(p => p.Id == id)
        .Select(p => new ProductDescription(p.Id, p.Name, p.Description))
        .FirstOrDefaultAsync(ct);

    if (row is not null)
    {
        // 60 seconds is the staleness budget for text, not for price.
        _cache.Set(key, row, TimeSpan.FromSeconds(60));
    }
    return row;
}

public async Task<decimal> PriceAsync(string sku, int quantity, CancellationToken ct)
{
    decimal unit = await _db.Products.AsNoTracking()
        .Where(p => p.Sku == sku)
        .Select(p => p.UnitPrice)
        .SingleAsync(ct);
    return unit * quantity;
}
```

The query filter from Day 5, or an explicit TenantId predicate, still applies. The cache key repeats the tenant so a missing filter is not the only line of defense. PriceAsync hits SQL on the create path. It does not read the description cache.

What to say in 20 minutes

1. Minute 0 to 2. One team, the order assumptions, a 60-second budget for description text only.
2. Minute 2 to 6. Catalog GET and the order POST. The price is computed on the server from SQL.
3. Minute 6 to 10. Two schemas, the keyset query, the covering index, why OFFSET loses.
4. Minute 10 to 14. Orders are read-your-writes. Descriptions may lag 60 seconds per instance. Prices are not cached.
5. Minute 14 to 17. A cache key without the tenant is a leak. SQL down is 503. A useless cache is removed.
6. Minute 17 to 20. You refuse the split, the replica, and the cache on the command. You name the condition that would reopen the split.

Interview trap

Caching the price inside the description object and then claiming the order total is current. A cache key of only the product id. Output caching an authenticated page without varying on the tenant. Splitting a service because the word catalog sounds like a bounded context, before you can name the data owner and the failure. Using OFFSET because the first page looked fast.

Say this

Orders stay on the primary, uncached, with a keyset list. Catalog text can sit in a per-instance cache for a staleness budget I state, and the price for a new order comes from SQL. I keep one deployable until a team, a release train, or a measured fight forces a cut. The cache key includes the tenant.

Part B, Day 4 — Nightly file ingest

Outcome. You can take a nightly file of about a million rows from blob storage into SQL in chunks, make a retry safe, keep the HTTP API responsive, and show the partner which lines failed.

Time box. About 3.5 hours. Draw the four hops without code. Then write the chunk transaction and the status API. Spend a full hour on the failure where one chunk is poison and the other chunks must continue. Speak the twenty-minute version once at a normal pace and once with someone asking where the file is parsed.

The prompt

A partner drops a CSV in storage every night, about a million rows. You validate, load, and report failures. The order API must stay inside its latency assumption while this runs. The mistake the panel is listening for is parsing the million rows on a request thread or inside the single function invocation that saw the blob.

Assumptions

The file is about a million data rows, which is the problem statement, not a platform limit. A chunk of 5,000 rows is the starting size you will change after you look at log growth and lock time. The business wants the file in a terminal state by a deadline you agree with them, and you alert if it is not. The API's interactive budget is unchanged: read p95 under 300 milliseconds at about 50 requests a second. The import does not have to finish in one second. It has to finish in the night window without taking the API with it. You look up the function timeout for the plan you run. You do not memorize it. You size the chunk so one invocation finishes with room to spare.

API

The partner does not upload through the order POST. They drop the blob, or an internal admin calls a small POST /imports that records a blob name you already accepted into the container. GET /imports/{fileId} returns the status and the error count. GET /imports/{fileId}/errors returns the bad lines, paged with a keyset on line number. The API never opens the file. 404 is a file id the caller cannot see. 409 is a second registration of a hash you have already merged, if that is the rule you documented.

Data

```
CREATE TABLE ingest.Files
(
    FileId       uniqueidentifier NOT NULL CONSTRAINT PK_Files PRIMARY KEY,
    TenantId     uniqueidentifier NOT NULL,
    BlobName     nvarchar(400)    NOT NULL,
    ContentHash  binary(32)       NOT NULL,
    Status       nvarchar(32)     NOT NULL,
    CreatedUtc   datetime2        NOT NULL
);
```

```
CREATE UNIQUE INDEX UX_Files_Hash_Merged
    ON ingest.Files (ContentHash)
    WHERE Status = N'Merged';

CREATE TABLE ingest.FileChunks
(
    FileId    uniqueidentifier NOT NULL,
    RowStart  int              NOT NULL,
    RowCount  int              NOT NULL,
    CONSTRAINT PK_FileChunks PRIMARY KEY (FileId, RowStart)
);

CREATE TABLE ingest.FileErrors
(
    FileId      uniqueidentifier NOT NULL,
    LineNumber  int              NOT NULL,
    Reason      nvarchar(400)    NOT NULL,
    CONSTRAINT PK_FileErrors PRIMARY KEY (FileId, LineNumber)
);

CREATE TABLE ingest.ProductBatch
(
    FileId     uniqueidentifier NOT NULL,
    TenantId   uniqueidentifier NOT NULL,
    Sku        nvarchar(64)     NOT NULL,
    Name       nvarchar(200)    NOT NULL,
    SourceHash binary(32)       NOT NULL
);
```

The chunk primary key is the idempotency key for a range. A retry of the same file id and row start finds the row and stops, or it runs an upsert that is itself idempotent and then inserts the marker. The filtered unique index stops a second successful load of the same bytes. A failed file can be replayed after a fix because the filter only covers Merged. ProductBatch is staging. It is not the table the API reads for a product page.

Consistency

Each chunk is one transaction: the set-based upsert into the real table, the error rows for that range, and the FileChunks marker. You do not hold one transaction open for a million rows. You do not call SaveChanges per row. A million tracked entities will miss the window and keep the log and the locks busy while the API is trying to place orders.

The upsert is idempotent on (TenantId, Sku). A replay of a chunk writes the same result. MERGE is a sketch you can defend for a single worker that already holds the chunk key. It has sharp edges with concurrency and triggers, so you are equally happy to write an UPDATE of the matching rows and an INSERT of the missing ones. Say that you will review the plan either way. The API's product read can see a chunk as soon as that chunk commits. That is acceptable for a nightly catalog load. If the business wants all-or-nothing visibility, you load into a staging table and swap a partition or flip a version column at the end. That is a bigger mechanism. You name it only if they ask for a point-in-time cutover. The default you ship is chunk-at-a-time visibility and a file status that says Processing until the last chunk lands.

Failure

A bad line is a FileErrors row, not a failed file. The rest of the chunk commits. A poison chunk, meaning a bug or a data shape that throws every time, dead-letters that chunk message. The other chunks continue. The file status stays Processing until a completion check sees the gap, then it becomes CompletedWithErrors or Failed, and the status payload lists the missing range. An alert fires if the file is not terminal by the deadline. A retry of a successful chunk does not double-insert, because of the chunk key and the idempotent upsert. A function timeout, whatever the current number is for the plan, is handled by making the chunk smaller, not by raising the timeout until a million-row parse fits.

The shared database can still stall the API if a chunk escalates locks or floods the log. You start with short transactions, row locks, a staging schema, and a worker identity. If you measure lock waits on the order workload during the window, you move staging to its own database. You do not start with a second server because a diagram had two cylinders. You also do not run the import on the App Service request thread.

Operations

Blob created raises an Event Grid event, or a small function notices the blob. That function inserts ingest.Files and enqueues one control message. It does not read the million rows. A splitter reads the blob only far enough to emit chunk messages: file id, blob name, row start, row count. Workers on the isolated worker pull from a Service Bus queue, peek-lock, and complete after the chunk transaction commits. The worker's managed identity can read the container and write the ingest and catalog tables it owns. It cannot alter. It is not app_orders_runtime. The API identity can select file status and errors for the tenant. The dashboard is files not terminal, dead-letter depth on the chunk queue, and SQL lock waits during the window. You prove the retry on a small file before you talk about the million.

The tradeoff refused

You refuse to parse the file in the blob trigger. You refuse EF per row. You refuse one transaction for the whole file. You refuse to stop the other 99 chunks because one chunk is poison. You refuse to give the worker the API's SQL login. You refuse a second database until a measurement says the shared log or the shared locks are the problem.

Sketch

```
public sealed record FileChunkMessage(Guid FileId, string BlobName, int RowStart, int RowCount);

[Function("RegisterFile")]
public async Task Register([EventGridTrigger] EventGridEvent evt, CancellationToken ct)
{
    // Insert ingest.Files as Received and send one control message.
    // Do not open the CSV here.
}

[Function("ImportChunk")]
public async Task Import(
    [ServiceBusTrigger("file-chunks", Connection = "ServiceBus")]
    ServiceBusReceivedMessage message,
```

```
    ServiceBusMessageActions actions,
    CancellationToken ct)
{
    var chunk = JsonSerializer.Deserialize<FileChunkMessage>(message.Body);
    if (chunk is null)
    {
        await actions.DeadLetterMessageAsync(message, deadLetterReason: "BadPayload");
        return;
    }

    if (await ChunkExists(chunk.FileId, chunk.RowStart, ct))
    {
        await actions.CompleteMessageAsync(message);
        return;
    }

    await using var tx = await _db.Database.BeginTransactionAsync(ct);
    await BulkUpsertRange(chunk, ct); // set-based, idempotent on (TenantId, Sku)
    _db.FileChunks.Add(new FileChunk(chunk.FileId, chunk.RowStart, chunk.RowCount));
    await _db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
    await actions.CompleteMessageAsync(message);
}
```

Turn auto-complete off for that function when you settle the message yourself, or you will complete twice and throw. host.json on the isolated worker is where that flag lives. A transient SQL failure should throw or abandon, not dead-letter. A payload you cannot parse should dead-letter on the first try. Bulk upsert can be a bulk copy into ProductBatch for that file and range, then an UPDATE plus an INSERT into catalog.Products, then a delete of the staged rows, all in the chunk transaction. The sketch name BulkUpsertRange is the boundary. You do not need the bulk-copy API on the whiteboard unless they ask. You do need to say it is set-based.

What to say in 20 minutes

1. Minute 0 to 2. A million rows is the file. 5,000 is the chunk size you will tune. The API latency assumption does not move.
2. Minute 2 to 5. The API only reads status and errors. The file lands in the container.
3. Minute 5 to 9. Register, split, work. The register function does not parse. The queue carries ranges.
4. Minute 9 to 13. One transaction per chunk, set-based upsert, unique (FileId, RowStart), bad lines in FileErrors.
5. Minute 13 to 16. A poison chunk dead-letters. The other chunks continue. The file status shows the gap. Alert on the deadline.
6. Minute 16 to 20. Separate managed identity, short locks, no Migrate, no import on the request thread. A second database only after you measure lock waits. Prove retry on a small file.

Interview trap

Reading the whole blob in the trigger and retrying from byte zero after a timeout. SaveChanges per row. A chunk retry with no range key, so the million rows double. A poison chunk that fails the orchestrator and leaves the rest unprocessed. Sharing the API's SQL login with the worker. Quoting a function timeout from memory and building the design around that fiction.

Say this

The trigger records the file. Workers own chunks. Each chunk commits with a unique range key and a set-based upsert. Bad rows are queryable. A poison chunk dead-letters without stopping the others. The API only reads status, and the worker has its own identity. I look up the function timeout and I size the chunk under it.

Part B, Day 5 — Multi-tenant SaaS

Outcome. You can put many tenants in one Azure SQL database safely, and you can say the specific condition under which one tenant gets a database of its own.

Time box. About 3.5 hours. Write the filter and the unique index from memory. Write the cross-tenant test in words until the expected status code is 404. Then the dedicated-database story, including the migration cost, in under five minutes. The twenty-minute clock is the last hour, twice.

The prompt

The order API is becoming a SaaS product. Many customers share it. Design the tenancy. The panel wants to know both the default and the escape hatch. They will try to get you to start from a database per tenant. Do not.

Assumptions

Hundreds of small tenants, one team, the same order assumptions until a tenant actually arrives with volume. No tenant has a contractual residency requirement yet. You have not measured a noisy neighbor. If either of those becomes true, the escape hatch applies to that tenant, not to all of them. The tenant id comes from the Entra token, a claim you issued at provisioning, not from the body and not from a route value the caller can edit. A route that includes a tenant slug is fine as a hint, and the token still wins when they disagree.

API

The URLs stay /orders and /products. The tenant is ambient, from the token. A body field named tenantId is ignored. An admin API, if you need one, is a separate role and a separate policy, and it still takes the tenant as an explicit argument the admin is allowed to pass, checked against an allow list, not as a silent override on the customer route. GET of another tenant's id returns 404. POST that would write another tenant's id writes the token's tenant instead, or it returns 400 if you would rather reject the mismatched body. Pick one and test it. Ignoring the field is the choice that matches the rest of this guide.

Data

```
CREATE UNIQUE INDEX UX_Orders_Tenant_Number
    ON orders.Orders (TenantId, Number);

-- Every tenant-owned table has TenantId.
-- Every unique business key includes TenantId.
-- The leading column of the hot index is TenantId.

CREATE TABLE tenants.Directory
(
    TenantId    uniqueidentifier NOT NULL CONSTRAINT PK_Directory PRIMARY KEY,
    Name        nvarchar(200)    NOT NULL,
```

```
    DatabaseKey nvarchar(64)     NOT NULL  -- 'shared', or a dedicated key
);
```

DatabaseKey names a connection. The connection string itself lives in Key Vault or in an app setting the managed identity can read. It does not live in the directory table. Most rows say shared. A dedicated key points at that tenant's database. The application resolves the connection per request after it knows the tenant, and the DbContext is scoped to the request.

Consistency

Inside one tenant, Day 2's transaction is unchanged. Across tenants there is no transaction, because there is no cross-tenant business operation in this product. A global query filter is a safety net. It is not the only control. The authorization policy checks the caller. The query takes the tenant from the context. A test proves the cross-tenant read fails. Raw SQL bypasses the filter, and IgnoreQueryFilters bypasses it. A review rule says a raw SQL string names TenantId in the predicate, and a worker that has no ambient tenant must set one on purpose.

Row-level security in Azure SQL is the extra net if you want the engine to enforce the tenant even when a query forgets the predicate. The session context is set when the connection opens, from the same tenant id as the filter. A bug that sets the wrong context and a bug that trusts the body are still bugs you test for. RLS does not replace the token. A background worker has no user token. It sets the session context from the message, which carries the tenant id that was written at commit time, and it uses a SQL login that is still not a database owner.

Failure

The failure you design for is a cross-tenant read or write. The status code is 404 on the read. The test uses tenant A's token and tenant B's order id. The failure you operate is a noisy neighbor: lock waits or CPU that you can show on one tenant's queries. You do not guess a DTU or a vCore number in the interview. You say you would look at Query Store and the resource metrics, and a tenant who shows up there is a candidate for a dedicated database. A missed migration on that dedicated database is a drift incident. That is the failure mode of the escape hatch, and you name it before you offer the hatch.

Operations

One Azure SQL database is the default, with the tenant column, the filter, the index, and the test. Backups restore everyone together, which is acceptable until a tenant needs their own restore point. The pipeline applies migrations to the shared database and, when dedicated databases exist, to each of them. A inventory of DatabaseKey values is part of the release. The app identity can read the directory and can touch tenant data only through the runtime role. Provisioning a tenant is an insert in the directory and an Entra app registration or an app role assignment, done by a workflow, not by a row somebody typed in production.

Moving one tenant out is a copy, a catch-up, and a cutover. You can put the tenant in a read-only window, copy the rows, apply the migrations to the new database, point Directory.DatabaseKey at the new key, and then allow writes. Dual-write is the alternative if you cannot take the window, and it is more moving parts

than this scale needs. You say which one you would run. You do not build a shard map for hundreds of small tenants on the first drawing.

The tradeoff refused

You refuse a database per tenant as the default. You would multiply every migration, every connection leak, and every forgotten index by the tenant count, and the team is one team with hundreds of small customers. You refuse to trust the body. You refuse a unique index on the order number alone, because numbers then collide across tenants or, if you catch the collision, you leak that a number exists. You refuse to let the filter be the only control.

Sketch

```
public sealed class OrdersDbContext : DbContext
{
    private readonly ITenantContext _tenant;
    public Guid TenantId => _tenant.TenantId;

    public OrdersDbContext(DbContextOptions<OrdersDbContext> options, ITenantContext tenant)
        : base(options)
    {
        _tenant = tenant;
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Order>().HasQueryFilter(o => o.TenantId == TenantId);
        modelBuilder.Entity<Order>().Property(o => o.RowVersion).IsRowVersion();
        modelBuilder.Entity<Order>().HasIndex(o => new { o.TenantId, o.Number }).IsUnique();
    }
}

-- Optional engine-side net. The app still sets the tenant from the token.
CREATE OR ALTER FUNCTION orders.fn_tenant(@TenantId uniqueidentifier)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS ok
    WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS uniqueidentifier);
```

The filter reads a property on the context so EF evaluates the current tenant per query. A scoped DbContext and a scoped tenant context are required. A singleton context would pin the first tenant and would also be illegal for EF's lifetime rules. The connection interceptor that sets SESSION_CONTEXT runs when the connection opens, with a parameter, not with the tenant id concatenated into the SQL text.

```
[Fact]
public async Task Tenant_a_cannot_read_tenant_b()
{
    Guid id = await SeedOrder(tenantB);
    using var scope = factory.WithToken(tenantA);
```

```
    HttpResponseMessage response = await scope.Client.GetAsync($"/orders/{id}");
    Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
}
```

What to say in 20 minutes

1. Minute 0 to 2. Hundreds of small tenants, one team, no residency clause yet, token wins over the body.
2. Minute 2 to 5. URLs do not take a tenant from the caller. 404 hides the other tenant. Admin is a separate policy.
3. Minute 5 to 9. TenantId on every table, unique keys include it, hot indexes lead with it, directory row points at shared.
4. Minute 9 to 13. Filter plus authorization plus a test. Raw SQL is a review rule. RLS is the extra net, session context set from the same id.
5. Minute 13 to 16. Noisy neighbor is a measurement, not a guess. Dedicated database is that tenant only.
6. Minute 16 to 20. The cost of the hatch: a migration per database, a cutover, drift if you miss one. You refuse the hatch as the default.

Interview trap

Trusting tenantId from JSON. A unique index on Number alone. Returning 403 for another tenant's id and confirming the row exists. A singleton DbContext. Believing the query filter covers FromSqlRaw. Starting from a database per tenant because a slide said isolation. Quoting a vCore number you do not have.

Say this

One Azure SQL database. TenantId comes from the token. The unique index and the hot index include it. A filter, a policy, and a cross-tenant test that expects 404. A dedicated database is a named exception for residency or for a neighbor I measured, and every migration then runs once per database.

Part B, Day 6 — Notification fan-out

Outcome. You can take the order outbox and fan it out to email, SMS, and push with Service Bus, an idempotent consumer, and a dead-letter path that does not roll back the order.

Time box. About 3.5 hours. Redraw Day 2's transaction in ten minutes. Then the topic, the delivery row, and the crash after the provider accepts the message. Spend the last hour speaking the twenty-minute version, and answer a poison-message interruption without deleting the order.

The prompt

When an order is placed, the customer may get an email, an SMS, and a push. Each channel fails on its own. A retry must not send a second email. Design the fan-out. The order API from Day 2 stays the system of record.

Assumptions

One OrderPlaced message per committed order. The write rate sits inside the existing assumption of about 50 requests a second at peak, and most of those requests are reads, so the mail volume is the order-create slice of that number, not a new million-message stream. Email within a minute remains the target you are willing to say out loud. SMS and push use the same target unless the business says one of them can lag longer. The provider can take an idempotency key. If a provider cannot, you say that the remaining duplicate window is the crash after their accept and before your commit, and you pick a provider call that is safe to repeat or you record the send more carefully. You do not invent a Service Bus quota.

API

There is no public POST /notify that the partner can spam. The notification is a consequence of the order commit. Support gets GET /orders/{id}/notifications, a list of channel, state, and last error, paged if you must, from a table the API can read. The customer does not get a webhook in this slice. If they ask for one, it is another subscription with the same dedupe rules, and the receiver still has to dedupe. 202 is the wrong code for the original POST /orders. That call remains 201, because the order is created. Mail is not the definition of created.

Data

```
CREATE TABLE orders.NotificationDeliveries
(
    Channel        nvarchar(16)     NOT NULL,
    MessageId      nvarchar(128)    NOT NULL,
    OrderId        uniqueidentifier NOT NULL,
    TenantId       uniqueidentifier NOT NULL,
    State          nvarchar(16)     NOT NULL, -- Sending, Sent, Failed
    AttemptCount   int              NOT NULL,
    UpdatedUtc     datetime2        NOT NULL,
```

```
    CONSTRAINT PK_NotificationDeliveries PRIMARY KEY (Channel, MessageId)
);
```

The primary key is the dedupe. Channel plus the Service Bus message id, which is the outbox id from Day 2, so a redelivery of the same publish is the same row. A second business event, such as OrderShipped, is a new outbox id and a new delivery. Do not dedupe on order id alone or you will drop the shipping notice.

Consistency

The order transaction does not include the provider. Day 2 already committed the outbox. The relay publishes one message to a Service Bus topic. Subscriptions named email, sms, and push each get a copy, each with its own peek-lock and its own dead-letter queue. A SQL filter on a subscription is how you skip SMS for an order that has no phone number, using a property the relay sets on the message. The filter is a convenience. The consumer still checks the payload.

The consumer's rule: insert the delivery row as Sending, call the provider with an idempotency key equal to the message id, mark the row Sent, then complete the Service Bus message. The unique key stops two workers. Peek-lock should already allow only one owner, and a lock that expires while the provider is slow is how a second worker appears. The second insert hits the primary key. If the row is Sent, the second worker completes the message and does not call the provider. If the row is Sending and the lease you stored in UpdatedUtc is still fresh, the second worker abandons or simply returns so the lock can expire, and the first worker remains the owner. If Sending is older than the lease you chose, the second worker may take over, and the provider idempotency key is what prevents a second email when the first call actually succeeded.

That lease interval is your choice, aligned with the lock duration you configured on the subscription. You look up how lock renewal works in the SDK you are on, and you renew a lock for a slow provider instead of hoping the default covers it. You do not quote a default duration from memory.

Failure

A transient provider error throws or abandons. Service Bus redelivers until the max delivery count you set on the subscription. A small count is a team choice so a poison message reaches the dead-letter queue during the incident rather than the next day. You read the portal default when you create the subscription, and you set the value on purpose. A malformed payload, an order id that does not exist, or an address the provider says is permanently invalid is poison. You dead-letter it with a reason. You do not abandon it in a loop. You do not delete the order. You do not mark the order failed because the SMS failed and the email succeeded. Each channel has its own row. An alert on dead-letter count greater than zero is the page. A replay after the fix uses the same message id so the delivery row still dedupes.

Operations

The consumers are Azure Functions on the isolated worker, one function per subscription or one function with the subscription name bound, whichever your team will actually find in the portal at night. Managed identity reaches Service Bus and SQL. The SQL user app_orders_notify may insert and update NotificationDeliveries and may select the order fields it needs to render the message. It may not alter tables and it may not delete orders. The API user is not this user. Auto-complete is off when the function settles the

message itself. The dashboard adds, per subscription, the active message count, the dead-letter count, and the age of the oldest message. Outbox age from Day 2 is still there, because a stuck relay never reaches this topic.

The tradeoff refused

You refuse one queue that the email worker forwards to SMS. A failure in the first consumer then blocks the other channels, and you have built an ordering nobody asked for. You refuse receive-and-delete. You refuse to complete the message before the provider call returns. You refuse to depend only on Service Bus duplicate detection. You refuse a distributed transaction. You refuse to send the three channels inside the order request. A topic with three subscriptions is the fan-out. If you only have email, one queue is enough, and the topic is what you add when the second channel arrives. That is a fair first slice, and you say so.

Sketch

```
{
  "version": "2.0",
  "extensions": {
    "serviceBus": {
      "autoCompleteMessages": false
    }
  }
}
```

```
[Function("SendEmail")]
public async Task Send(
    [ServiceBusTrigger("order-events", "email", Connection = "ServiceBus")]
    ServiceBusReceivedMessage message,
    ServiceBusMessageActions actions,
    CancellationToken ct)
{
    if (!TryReadOrder(message, out Guid orderId))
    {
        await actions.DeadLetterMessageAsync(message, deadLetterReason: "BadPayload");
        return;
    }

    var delivery = new NotificationDelivery
    {
        Channel = "email",
        MessageId = message.MessageId,
        OrderId = orderId,
        State = "Sending",
        AttemptCount = 1,
        UpdatedUtc = DateTime.UtcNow
    };

    try
    {
        _db.NotificationDeliveries.Add(delivery);
        await _db.SaveChangesAsync(ct);
```

```
    }
    catch (DbUpdateException ex) when (IsUniqueViolation(ex))
    {
        _db.ChangeTracker.Clear();
        if (await AlreadySent("email", message.MessageId, ct))
        {
            await actions.CompleteMessageAsync(message);
            return;
        }
        await actions.AbandonMessageAsync(message);
        return;
    }

    await _email.SendAsync(orderId, idempotencyKey: message.MessageId, ct);
    delivery.State = "Sent";
    delivery.UpdatedUtc = DateTime.UtcNow;
    await _db.SaveChangesAsync(ct);
    await actions.CompleteMessageAsync(message);
}
```

A permanent provider rejection is a catch you write next to this method: mark the row Failed, dead-letter, do not throw. A transient rejection throws before the complete, so the lock rolls back to an abandon and the Sending row is what the next attempt finds. The next attempt's unique-key path has to be allowed to continue a Sending row whose owner died. The sketch above abandons on any existing row that is not Sent, which is safe when the first worker is still running and slightly slow when it died. Say the lease takeover in the interview even if you do not write every branch on the board. The important line is the provider idempotency key.

What to say in 20 minutes

1. Minute 0 to 2. Fan-out is a consequence of OrderPlaced. Volume stays inside the order assumption. One minute is still the mail target.
2. Minute 2 to 5. No public notify API. Support reads delivery state. POST /orders is still 201.
3. Minute 5 to 8. Topic order-events, three subscriptions, each with its own dead-letter. Delivery primary key is channel plus message id.
4. Minute 8 to 13. Insert Sending, call the provider with the message id, mark Sent, then complete. A duplicate key means already done or still in flight.
5. Minute 13 to 17. Transient abandons. Poison dead-letters. The order stays. Alert on dead-letter depth. Replay keeps the message id.
6. Minute 17 to 20. Isolated worker, its own SQL user, auto-complete off, peek-lock. You refuse a chain of queues and a two-phase commit.

Interview trap

Deduping on order id and dropping later events. Completing the message and then calling the provider. Receive-and-delete. One shared queue with a hand-rolled fan-out that stops all channels when email throws. Treating a dead letter as a reason to roll back the order. Quoting a lock duration or a duplicate-detection window you have not checked. Leaving auto-complete on and also calling Complete.

Say this

The outbox publishes once to a topic. Each channel is a subscription with its own lock and its own dead-letter. The consumer inserts a delivery row keyed by channel and message id, calls the provider with that id, and only then completes. A poison message waits in the dead-letter queue. The order stays placed.

Part B, Day 7 — Rate limiting and the public gateway

Outcome. You can put the ASP.NET Core rate limiter on the order API, return 429, and explain who enforces the limit when more than one instance is running.

Time box. About 3.5 hours. Write the policy from memory, including the partition key. Then the two-instance arithmetic on a whiteboard: a per-instance window is not a global window. Spend the last hour on where a gateway fits, without naming a price or a quota you would have to look up.

The prompt

Partners call the order API from the public internet. Design how you stop one partner from consuming the database. Say what the caller receives, and say which hop enforces the rule.

Assumptions

The friendly load is still about 50 requests a second at peak for the whole API, and the limiter exists for the unfriendly case. A policy of 100 requests per minute per client application, per instance, is a number you chose so you can talk about it. It is not an Azure default and it is not a capacity plan. You change it after a test, and you write the chosen number down next to the client contract. Anonymous traffic, if you have any, is partitioned by a coarser key and a smaller budget. The read p95 target stays under 300 milliseconds for callers inside the budget.

API

Every partner route uses the policy, including POST /orders and the GETs. A rejected call returns 429 with a Retry-After header when the limiter provides a retry interval. The body is a short problem document, not a stack trace. 429 is not 503 and not 500. A client that retries immediately is a client bug, and your Retry-After is the hint. The order idempotency key still matters, because a retry after 429 must not create a second order if the first attempt actually committed and the 429 was a later duplicate. In practice a limited request is rejected before the handler. Say that. The dangerous retry is the timeout, which Day 2 already handles.

Authentication still returns 401 and authorization returns 403. A caller who fails authentication does not get a 429 instead. A flood of bad tokens is a reason to limit before you spend work on them, which is the gateway's job as well as a coarse limiter at the edge of the app.

Data

The built-in limiter keeps its counters in memory in the process. There is no SQL table in the first slice. The partition key is the Entra application id, the azp claim or the appid claim, not a field in the JSON and not the raw IP address of a partner who shares a NAT with the rest of their company. IP is the key only for traffic that has no token. A successful order still writes the Day 2 tables. The limiter does not become a second source of truth for idempotency.

Consistency

A fixed window is easy to explain and allows a burst at the boundary between two windows. A sliding window or a token bucket smooths that burst and is still an in-process structure. You start with a fixed window, queue limit zero, so an over-budget call fails immediately instead of occupying a thread. A concurrency limiter is the other knob, for a whole instance, so a flood cannot schedule unbounded work even if the per-client windows are wide. Both can be on. The endpoint policy and a global limiter both run.

Two instances means two independent windows. A client whose policy is 100 per minute per instance can do about 200 per minute if the load balancer spreads them evenly, and more if they pin to one instance and you misread the metric. That is acceptable only if you meant it. A shared budget across instances lives in a place that sees every call. That place is the gateway in front: Azure Front Door, Azure API Management, or a reverse proxy you already operate. You look up that product's current limits when you pick it. You do not recite them. The app limiter stays, because a caller who reaches an instance directly, or a misconfigured route that bypasses the gateway, must still be unable to melt SQL.

Failure

The limiter fails closed in the sense that a request over budget is rejected. A process recycle resets the in-memory window, so a deploy gives everyone a fresh budget. That is fine. A limiter bug that rejects everyone is an incident: you would rather have a config flag to disable the policy than a code change, and you watch 429 rate next to 500 rate so you can tell a limit from a crash. SQL is not your rate limiter. If SQL is the thing returning errors under load, the limiter was set too high or it was not on the right route. The health check stays cheap and is excluded from the partner policy, or a busy partner can push the instance out of rotation by starving the probe. Confirm the probe path is anonymous and not limited by the partner key.

Operations

The gateway is the shared door. It terminates TLS, it can reject traffic that has no token if you configure it to, and it enforces the budget that is in the contract. The app always validates the token itself. A gateway that checked the token is a performance help, not a trust boundary you rely on alone. App Service keeps ARR affinity off. You do not turn affinity on to make an in-memory rate limit look global. That would pin a partner to one instance and make the other instance idle, and it would still be wrong the day you scale. Managed identity is unchanged. The policy numbers live in configuration so a change is not a recompile, and the configuration is still a choice you review.

The tradeoff refused

You refuse a limiter that only exists at the gateway. You refuse a limiter that only exists in one process and is described to the partner as a global contract. You refuse to build a new ASP.NET gateway service for this. If the estate already has YARP in front for a migration, a limit can live there too, and the app still has its own. You refuse a custom distributed limiter in Redis as the first design. You add a shared store only after the gateway cannot express the rule. You refuse to queue limited requests inside the app and call that protection. A queue of waiters is how a flood becomes a thread-pool incident.

Sketch

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.OnRejected = async (context, ct) =>
    {
        if (context.Lease.TryGetMetadata(MetadataName.RetryAfter, out TimeSpan retryAfter))
        {
            context.HttpContext.Response.Headers.RetryAfter =
                Math.Ceiling(retryAfter.TotalSeconds).ToString(CultureInfo.InvariantCulture);
        }
        await context.HttpContext.Response.WriteAsync("Too many requests.", ct);
    };

    options.AddPolicy("per-client", httpContext =>
    {
        string key = httpContext.User.FindFirst("azp")?.Value
            ?? httpContext.User.FindFirst("appid")?.Value
            ?? "anonymous";

        return RateLimitPartition.GetFixedWindowLimiter(
            key,
            _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 100, // team policy, per instance, not an Azure default
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0
            });
    });
});

app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();

[EnableRateLimiting("per-client")]
[HttpPost("/orders")]
public Task<IActionResult> Create(CancellationToken ct) => /* Day 2 */;
```

```
# Policy you chose. Change it in config, review it like a contract.
RateLimit:
  PermitLimit: 100
  WindowSeconds: 60
  QueueLimit: 0
```

The middleware order puts authentication first so the partition key can be the application id. A flood of unauthenticated calls still spends the authentication middleware. The gateway is what sheds that flood before it reaches the instance. A global concurrency limiter, with a permit count you set from a load test rather than from a guess you memorize, is the backstop on the instance. Exclude the health path.

What to say in 20 minutes

1. Minute 0 to 2. Public partners, the 50 request assumption for friendly traffic, and a per-client policy you label as a choice.
2. Minute 2 to 6. 429 and Retry-After. 401 and 403 stay what they are. The handler does not run when the limit rejects.
3. Minute 6 to 10. In-memory fixed window, queue limit zero, partition key from the token. IP only when there is no token.
4. Minute 10 to 14. Two instances mean two budgets. The contract's shared number is enforced at the gateway. The app limit stays for bypass.
5. Minute 14 to 17. Recycle resets the window. Health probes are not in the partner policy. SQL is not the limiter.
6. Minute 17 to 20. You refuse gateway-only, app-only sold as a global contract, a homegrown gateway, and a Redis limiter as the opening move.

Interview trap

Returning 500 when the limit trips. Using the JSON tenant or the client-supplied api key header as the only partition key when the token already has an app id. Turning on ARR affinity so the memory counter looks stable. Describing a per-instance window as a global guarantee. Quoting a Front Door or API Management number you have not opened the document for. Rate limiting by queuing thousands of requests.

Say this

The app uses the built-in fixed window, partitioned by the Entra application id, and it returns 429. That window is per instance. The shared contract is enforced at the gateway in front, and the app still limits so a bypass cannot melt SQL. I look up the gateway's limits when I pick the product. I do not put a queue of waiters in the process.

Part B, Day 8 — A URL shortener

Outcome. You can design a redirect service in ASP.NET Core: unguessable keys, a cache in front of a primary-key lookup, and a plan for one code that suddenly gets all the traffic.

Time box. About 3.5 hours. This day is the deep one. Write create and resolve from memory, including the collision retry and the 302. Then spend a full hour on the hot key only: what is cached, where, and what a second instance changes. The twenty-minute version should still leave five minutes for the hot key. If you run out of time, you cut the analytics story, not the redirect.

The prompt

Design a service that stores a long URL and redirects a short code to it. Creation is rare. Redirect is the hot path. One code may become popular. You are not designing a general pastebin, and you are not counting every click in the redirect transaction.

Assumptions

These numbers are for this design, and they are assumptions, separate from the order API. About 200 redirects a second at peak, about 5 creates a second, a cache-hit p95 under 50 milliseconds, a cache-miss p95 under 300 milliseconds. One region. Links are unlisted: guessing a code should not be practical. Targets change rarely. A 10-second cache of a target is an acceptable staleness budget for a redirect, and you label those 10 seconds as your choice. Expiry is a timestamp on the row, not a cache feature. You are not promising a click count that is exact to the second.

API

POST /links takes an absolute http or https URL and returns 201 with the code and the short URL. 400 when the target is missing, relative, or a scheme you do not redirect to. GET /{code} returns 302 with a Location header, 404 when the code is unknown, and 410 when the row exists and is expired. You use 302, not 301. A 301 is cached hard by clients, and you will not be able to expire or change the target. The redirect target is the value you stored, never a URL from the query string. That is how this stays a shortener and does not become an open redirect. A stats route, if you add one, is not on the redirect path and can be minutes behind.

Data

```
CREATE TABLE links.Links
(
    Code       varchar(12)      NOT NULL CONSTRAINT PK_Links PRIMARY KEY,
    TargetUrl  nvarchar(2048)   NOT NULL,
    CreatedUtc datetime2        NOT NULL,
    ExpiresUtc datetime2        NULL
);
```

The code is the primary key. It is 7 characters from a 62-symbol alphabet, generated with RandomNumberGenerator, not with a guessable identity column. Seven characters is a choice: the space is 62 to the 7th, about 3.5 trillion codes, which is mathematics, not an Azure limit. You still handle a primary-key collision, because the check is cheap and the unique index is the real guard. A sequential integer encoded into base62 is shorter to generate and easier to scan. You refuse it for unlisted links. Modulo bias exists if you take a random byte and reduce it modulo 62. For this sketch the bias is small, and if they push, you switch to rejection sampling: discard a byte at or above the largest multiple of 62 that fits in a byte. Say that you know the bias is there.

Consistency

Create is a single insert. There is no outbox, because nothing else must happen for the link to be real. A collision rolls back that insert and tries a new code, with a small attempt limit, then 503 if you somehow cannot find a free code. You will not hit that limit at 5 creates a second in a space this large. Redirect is a read. The cache is an optimization of a primary-key lookup. SQL remains the record. A cache hit can be at most the staleness budget behind an update or a delete. If you need a taken-down link to die everywhere immediately, you delete it from SQL and you accept that a per-instance cache can still redirect until the local entry expires, or you put a distributed cache in front and delete the key. Immediate global revocation is the requirement that justifies the distributed cache. A viral cat picture is not.

Failure

SQL down and a warm cache: hits still redirect, misses return 503. SQL down and a cold cache: 503. An unknown code is a 404, and you cache that negative answer for a short budget, for example 30 seconds, so a scanner walking random codes does not pin those misses forever and does not hammer SQL. The 30 seconds is your choice. The race is a code that was probed before it was created. With random 7-character codes that race is rare, and a short negative TTL is why you can live with it. A hot key is one code that draws a large fraction of the 200 redirects a second. A primary-key lookup of one row can take that rate. The failure is a stampede when the cache entry expires and every instance misses together, or a design that counts a click in the same transaction as the redirect and turns a read into a write hotspot on one row.

You keep a small local cache even if a distributed cache exists, with the 10-second budget, so the hottest code is served from the process. The distributed cache, if you add one, absorbs the misses of many instances. You do not take a distributed lock on the read path. One instance missing and querying SQL by primary key is cheap. If you measured a stampede, a single-flight lock inside the process is the first tool, and it only collapses the misses on that instance. Say the measurement before you add the lock.

Operations

The API runs on App Service, affinity off, more than one instance. The local cache is therefore a per-instance cache, and that is what you want for the hot key: each instance holds the popular code and does not coordinate. Managed identity reaches Azure SQL. The app user can insert and select links and cannot alter. There is no Function in the first slice. Click analytics, if required, is a sampled write or a message, consumed off the redirect path, and it is allowed to lose samples. The dashboard is redirect latency, cache hit rate, SQL

duration, and 404 rate. A sudden rise in 404 rate is a scanner or a bad deploy, and you want to see it. Health checks the database and is not redirected.

The tradeoff refused

You refuse a 301. You refuse a sequential public code. You refuse to increment a click counter in the redirect transaction. You refuse a distributed lock around every read. You refuse to generate the code on the client and trust it. You refuse to put the target URL in the query string and redirect there. You refuse a cache that has no tenant or owner only if this service is multi-tenant: the code itself is the secret, and the code is globally unique in this design, so the key is the code. If two tenants could claim the same code, the code would include a tenant, and this design does not do that. One global namespace of unguessable codes is the product.

Sketch

```
public static string NewCode(int length = 7)
{
    const string Alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    Span<char> chars = stackalloc char[length];
    Span<byte> bytes = stackalloc byte[length];
    RandomNumberGenerator.Fill(bytes);
    for (int i = 0; i < length; i++)
        chars[i] = Alphabet[bytes[i] % Alphabet.Length];
    return new string(chars);
}

app.MapPost("/links", async (CreateLink body, LinksDb db, CancellationToken ct) =>
{
    if (!Uri.TryCreate(body.TargetUrl, UriKind.Absolute, out Uri? uri)
        || uri.Scheme is not ("https" or "http"))
        return Results.Problem(statusCode: 400, title: "Target must be an absolute http(s) URL.");

    for (int attempt = 0; attempt < 5; attempt++)
    {
        var link = new Link { Code = NewCode(), TargetUrl = uri.AbsoluteUri, CreatedUtc =
DateTime.UtcNow };
        db.Links.Add(link);
        try
        {
            await db.SaveChangesAsync(ct);
            return Results.Created($"/links/{link.Code}", new { code = link.Code });
        }
        catch (DbUpdateException ex) when (IsUniqueViolation(ex))
        {
            db.ChangeTracker.Clear();
        }
    }
    return Results.StatusCode(StatusCodes.Status503ServiceUnavailable);
});

app.MapGet("/{code}", async (string code, LinkCache cache, CancellationToken ct) =>
```

```
{
    string? target = await cache.Resolve(code, ct);
    return target is null ? Results.NotFound() : Results.Redirect(target, permanent: false);
});
```

```
public async Task<string?> Resolve(string code, CancellationToken ct)
{
    if (_cache.TryGetValue(code, out CachedLink? cached))
        return cached!.Found ? cached.Target : null;

    var row = await _db.Links.AsNoTracking().FirstOrDefaultAsync(l => l.Code == code, ct);
    if (row is null || (row.ExpiresUtc is not null && row.ExpiresUtc <= DateTime.UtcNow))
    {
        _cache.Set(code, new CachedLink(false, null), TimeSpan.FromSeconds(30));
        return null;
    }

    // 10 seconds: staleness budget for a hot redirect, per instance.
    _cache.Set(code, new CachedLink(true, row.TargetUrl), TimeSpan.FromSeconds(10));
    return row.TargetUrl;
}

public sealed record CachedLink(bool Found, string? Target);
```

Results.Redirect with permanent false is a 302. The negative cache stores a record, not a null, because the memory cache is a poor place to store null and a missed null is indistinguishable from a miss. Span in NewCode avoids a heap buffer for a 7-byte draw. The modulo line is the one you replace with rejection sampling if they ask about bias. Do not do that replacement until they ask. The first version has to be correct about the unique index, the scheme check, and the 302.

What to say in 20 minutes

1. Minute 0 to 2. 200 redirects a second, 5 creates, hit under 50 milliseconds, miss under 300, all assumptions. Unguessable codes.
2. Minute 2 to 6. POST validates http(s). GET returns 302. 404 unknown, 410 expired. Never redirect to a query-string URL.
3. Minute 6 to 10. varchar code as the primary key. Random 7-character code, retry on unique violation. No sequential public id.
4. Minute 10 to 14. SQL is the record. Local cache, 10-second budget, negative cache at 30 seconds. Both numbers are choices.
5. Minute 14 to 17. A hot key stays in each instance's memory. No click update on the redirect. No distributed lock. SQL down serves hits only.
6. Minute 17 to 20. App Service, affinity off, managed identity, a SQL user that cannot migrate. Analytics is off the hot path or it is not in this slice.

Interview trap

A 301. An IDENTITY column as the public code. Trusting the client to pick the code. Redirecting to a URL supplied on the GET. Updating a hit counter on the only row everyone wants. A distributed lock on the read. Caching null and being unable to tell a 404 from a cold key. Quoting a Redis operations limit you have not measured.

Say this

The code is a random primary key, generated on the server, with a retry on collision. GET is a 302 to the stored http(s) URL. A per-instance cache absorbs the hot code for a staleness budget I state. SQL remains the record. I do not count clicks on the redirect, and I do not take a distributed lock to read one row.

Part B, Day 9 — Realtime order status

Outcome. You can choose polling or SignalR for order status, defend the choice with the open-screen count, and explain a scale-out backplane without quoting a product limit.

Time box. About 3.5 hours. Compute the poll rate on paper for three different watcher counts until the arithmetic is fast. Then the SignalR sketch and the backplane tradeoff. The twenty-minute version must include the sentence that SQL is still the record and the push is a hint.

The prompt

A customer page and an internal warehouse board both show order status. Status changes a handful of times over the life of an order. Design the update path. Pick polling, SignalR, or both, and say why.

Assumptions

The order API assumptions still hold for the write path. The new number is how many screens are open. Assumption for the first slice: dozens of customer screens, not tens of thousands, and a warehouse board that a shift of people keep open all day. A customer poll every 2 seconds is a product choice. Dozens of screens at that interval fit inside a budget like 50 requests a second. Twenty thousand tablets at a 2-second poll are 10,000 requests a second, which breaks the assumption, and that is the moment the warehouse stops polling. You label the watcher count. You do not invent a SignalR connection quota. When the count matters you open the current Azure SignalR Service SKU page.

API

GET /orders/{id} remains the source of truth and returns the status and the rowversion. The customer page and the partner poll it. The warehouse board opens a SignalR connection and joins a group for the orders on that bench, or one group per order. The hub method is Watch(orderId). The server checks the tenant and that this caller may see the order before it adds the connection to the group. A push message contains the order id, not the new status. The client calls GET. A push that carried the status could arrive out of order and paint an old state over a new one. The GET after the hint cannot. If the socket is down, the board falls back to the same GET on a slow timer, so the socket is not required to see the truth.

Data

No new table is required for polling. The status column and the index you already have for the point read are the data. SignalR groups live in memory on the connection host, not in SQL. You do not store sockets in a table. If you later need a missed-event replay, the order row is the replay. The client that reconnects calls GET and is current. That is simpler than an event log, and you add an event log only when a client must see the sequence rather than the latest status. This product needs the latest status.

Consistency

The status change commits in SQL first, in the same transaction as the order update and, if someone else must hear about it, the outbox row. After commit, the API notifies the group. A notify that runs before commit is how the client refetches and still sees the old status, then ignores the next push because it thinks it already handled that id. After commit, a failed notify is not a failed order. The client's poll or its next reconnect sees the row. The push is a hint. You refuse to make the hub the system of record.

You choose both, on purpose. Polling for the customer and the partner, because the count is small, the GET already exists, and App Service can stay with affinity off and no extra product. SignalR for the warehouse board, because those screens sit open and a 2-second poll from a wall of browsers is the load you do not want, and because a one-second hint is the point of the board. If the panel forces a single mechanism for the first release, you ship polling. The GET is the contract. The hub is an optimization you can add without changing the write path.

Failure

SQL down: the GET returns 503, and the hub has nothing true to say. You do not cache a status and serve it as current. SignalR Service down, or your hub down: the board falls back to polling, and the customer path never notices because it was already polling. A second API instance with in-process SignalR and no backplane drops events that were raised on the other instance, and a client connected to instance A never hears a notify executed on instance B. That is the scale-out bug. A client that fails the tenant check is not added to the group. A leaked group name that includes only the order id, without a tenant check on join, is a cross-tenant subscription. The check is in the hub method, not in the group name string alone.

Operations

The order API stays on App Service with ARR affinity off. Polling does not care which instance answers. SignalR in-process would care, because the connection lives on one instance. Sticky sessions would pin the socket, and they fight the affinity-off decision you already made for the API: scale-in and restarts drop pins, and a pinned hot board lands on one instance. You do not turn affinity on.

The backplane choice, at the level of a name and a tradeoff: Azure SignalR Service holds the client connections. The app server sends to the service, and the service fans out to the group. The app stays stateless and affinity stays off. You take a dependency and a network hop, and you look up the SKU when you know the connection count. The alternative is a Redis backplane: each app instance still holds its own WebSockets, and Redis spreads the group messages between instances. You operate Redis, and the connection count sits on App Service, which you measure rather than memorize. You pick Azure SignalR Service for the warehouse board so the API process is not the socket server. You do not quote a connection ceiling in the interview.

The tradeoff refused

You refuse the hub as the source of truth. You refuse sticky sessions as the scale-out plan. You refuse to notify before the SQL commit. You refuse a design where the only way to see status is the socket. You refuse

to poll 20,000 clients every 2 seconds and call it simple once the arithmetic is on the board. You refuse a second status column that only the hub writes.

Sketch

```
public sealed class OrderHub : Hub
{
    public async Task Watch(Guid orderId)
    {
        if (!await _orders.CallerMaySee(orderId, Context.User))
            throw new HubException("Not found.");
        await Groups.AddToGroupAsync(Context.ConnectionId, Name(orderId));
    }

    public static string Name(Guid orderId) => $"order:{orderId:N}";
}

// After the status transaction commits. The payload is a hint.
await _hub.Clients.Group(OrderHub.Name(order.Id))
    .SendAsync("statusChanged", new { order.Id }, ct);

// Program.cs, when the warehouse board is in this release.
builder.Services.AddSignalR().AddAzureSignalR();
```

AddAzureSignalR is the managed backplane. The app authenticates to it with a connection string held as a secret, or with the managed-identity setup in the current documentation. You say you will use managed identity and you will read the current setup, rather than pasting a key into a sketch from memory. The customer client does not reference the hub. It polls GET /orders/{id} on the 2-second timer while the page is visible, and it stops when the page is hidden. That last clause is how a few dozen open tabs do not become a few dozen forgotten tabs.

What to say in 20 minutes

1. Minute 0 to 3. Dozens of customer screens poll every 2 seconds, a labeled choice. The warehouse board is the screen that stays open. Write the request-rate arithmetic.
2. Minute 3 to 7. GET is the record. 200 with status and rowversion. The partner uses the same GET.
3. Minute 7 to 11. Polling for the customer. SignalR for the board. The event carries the id. The client refetches. Join checks the tenant.
4. Minute 11 to 15. Commit, then notify. A missed notify is healed by GET. The hub is not a database.
5. Minute 15 to 18. Affinity stays off. Azure SignalR Service holds the sockets. Redis is the backplane you would operate yourself, and then the sockets still sit on the app. You look up the SKU when the count is real.
6. Minute 18 to 20. If they force one mechanism, ship polling. Refuse sticky sessions and a socket-only status.

Interview trap

Sending the status in the push and letting the client paint it with no GET. Notifying before commit. Turning ARR affinity on and calling that a backplane. Forgetting the tenant check on group join. Quoting a connection limit from memory. Polling a population you have not multiplied by the interval. Making SignalR the only read path.

Say this

GET /orders/{id} is the status. Customers poll it while the page is open, at an interval I state, and I do the arithmetic before I keep that choice. The warehouse board uses SignalR, joins a group only after a tenant check, and treats the push as a hint to read. Azure SignalR Service is the backplane so the API can stay affinity off. I look up the SKU instead of inventing a connection limit.

Part B, Day 10 — A 45-minute mock

Outcome. You can hold a 45-minute design that combines the order API and the notification fan-out, and you can answer the four panel questions: the SQL role, the rollback, the second instance, and the poison message.

Time box. About 3.5 hours. The first hour is a silent redraw of Day 2 and Day 6 on one sheet, no notes. The next hour is the 45-minute clock once, alone. The last 90 minutes is the clock again, and you stop to answer each panel question in under two minutes when an imaginary interviewer interrupts. If you only do one run-through, do the interrupted one.

The prompt

Design the system that accepts an order from a partner, stores it, and notifies the customer by email and SMS. Partners retry. The warehouse may later want live status. You have 45 minutes. The panel will interrupt. Combine Day 2 and Day 6. Leave realtime as a sentence unless they pull it in, and if they do, use Day 9's choice: polling first, SignalR only for a board whose watcher count justifies it.

Assumptions

Say them in the first two minutes and write them where you can point: about 50 requests a second at peak, about 2 million orders a year, read p95 under 300 milliseconds, email and SMS within a minute, one region, one team, server-side pricing. Out of scope until they open it: a second region, a pricing service, a million-row nightly file, a database per tenant. If they change a number, stop and say what moves. At this number, one App Service and one Azure SQL database stay. The file ingest and the dedicated tenant database are other days. You can name them as the next review, not as boxes on this drawing.

API

POST /orders with Idempotency-Key. 201, replay 200, conflicting body 409, missing key 400. The body is sku, quantity, and ship-to. GET /orders/{id} is 200 or 404. GET /orders?cursor= is the keyset list. PATCH uses the rowversion and returns 409 on a lost update. GET /orders/{id}/notifications is the support view of email and SMS state. 401 and 403 stay on the auth middleware. 429 is Day 7, and you mention the limiter in one sentence if the word public is in the prompt: per-client fixed window in the app, shared budget at the gateway, no invented quota. Do not spend the mock designing the limiter unless they ask.

Data

orders.Orders with TenantId, rowversion, and the list index. orders.IdempotencyKeys unique on (TenantId, KeyValue) with the raw-body hash. orders.Outbox with the unpublished filtered index and the lease columns. orders.NotificationDeliveries unique on (Channel, MessageId). The topic is order-events. The subscriptions are email and SMS. Push is a third subscription you can add without a new publisher. The SQL users are app_orders_runtime for the API and app_orders_notify for the consumers. The pipeline migrator is not either of them.

Consistency

One transaction: order, idempotency row, outbox. After commit, the relay leases unpublished rows with UPDLOCK and READPAST, publishes with MessageId equal to the outbox id, then marks published. Each subscription inserts a Sending row, calls the provider with that message id, marks Sent, and completes the peek-lock. A duplicate publish is a duplicate key in the delivery table, not a second email. The API response does not wait for either provider. 201 means the order is durable.

Failure

Walk the four panel questions as failures, because that is how they will ask them. They are written in full below so you can practice the answer, not so you can read it in the room. In the mock you give the answer from the drawing.

What SQL role does the app use?

The API connects with its managed identity as app_orders_runtime. That user can select, insert, and update in the orders schema. It cannot alter, drop, or own the database. The notify functions use app_orders_notify, which can write delivery rows and read the order fields the template needs, and cannot delete orders. The pipeline uses a migrator identity that is allowed to apply the reviewed script and is not in the application connection string. The partner's Entra token is not a SQL login. It is how the API decides 401 versus 403 versus the tenant id. If you can only remember one sentence: the application login is not db_owner, and it is not the migrator.

How do you roll back?

The release expanded the schema. New tables and nullable columns are already there, and the previous bits do not need them. You swap the App Service slot back. You do not restore the database. A release that dropped a column or narrowed a type is not reversible by a swap, so that release is not the shape you shipped. Messages already on the topic are not recalled. Consumers accept this payload and the previous one for the overlap. If the consumer is the bad bit, you stop the relay with the flag or the function setting, you fix the consumer, and you replay. You do not delete orders to get back to a quiet topic.

What changes when a second instance is running?

The second API instance is normal, and affinity stays off. Idempotency is the unique key in SQL, so both instances can take the same retry and one of them loses the insert. An in-memory set of keys would have created two orders. The outbox lease skips rows the other relay locked, and a double publish still meets the delivery primary key. A per-instance rate limit is twice the budget unless the gateway holds the contract. A static cache of order status would be stale and split-brained, which is why GET hits SQL. SignalR is not in this slice. If they add the warehouse board, the second instance is why you want Azure SignalR Service rather than in-process sockets. Readiness must pass before the load balancer sends traffic, and readiness includes SQL, so a second instance that cannot reach the database does not take orders.

What do you do with a poison message?

A poison message is a payload you cannot parse, an order you cannot find, or an address the provider has permanently rejected. A timeout from the provider is not poison. You abandon transient failures until the max delivery count you set on that subscription, then the message dead-letters. You alert when the dead-letter count is above zero. You do not abandon forever, and you do not complete the message to make the queue look healthy. The order remains placed. Support can see the delivery row in Failed or can see the gap. After the fix, you replay the dead-letter with the same message id so the dedupe still holds. SMS dying does not roll back email and does not roll back the order.

Operations

App Service for the API, staging slot, Always On, affinity off, health on /health/ready. Functions on the isolated worker for the relay and for each subscription, auto-complete off where you settle the message. Managed identity throughout. The dashboard is request latency, SQL duration, outbox age, and dead-letter depth per subscription. The release is migrate, then deploy the slot, then smoke, then swap. The smoke creates an order with a key, retries it, and checks that one row exists. That smoke is the idempotency test you are willing to run in production against a canary tenant or a synthetic sku.

The tradeoff refused

In this mock you refuse, out loud, a pricing microservice, a two-phase commit, mail inside the request, receive-and-delete, an in-memory idempotency cache, Migrate on startup, and a shared SQL login in a high-privilege role. You also refuse to spend the forty-fifth minute on Kubernetes. If they ask what you would add next, the answer is the gateway limiter from Day 7 if the partners are untrusted, or the polling client from Day 9 if a screen needs status. The nightly file and a dedicated tenant database stay off this drawing until a number or a contract puts them on it.

Sketch

One sheet, which you should be able to redraw from memory. The boxes are the partner, App Service, Azure SQL, the relay function, the topic, two subscription functions, and the two providers. The arrow from App Service to SQL is labeled with the transaction. The arrow from the relay to the topic is labeled after commit. The arrow from a function to a provider is labeled with the message id. The identities sit under the boxes, three of them, plus Entra above the partner.

```
Partner -- Entra token --> App Service (affinity off, two instances)
                           |  one SQL transaction
                           v
                     Azure SQL
                       orders.Orders
                       orders.IdempotencyKeys
                       orders.Outbox
                       orders.NotificationDeliveries
                           |
            relay function, lease, after commit
                           v
                  topic order-events
```

```
                    |                |
                 email              sms
                    |                |
              provider, keyed by MessageId

SQL roles: app_orders_runtime, app_orders_notify, pipeline migrator
Rollback: slot swap, schema was expanded only
Second instance: unique keys, not memory
Poison: dead-letter the subscription, keep the order
```

What to say in 20 minutes

The mock is 45 minutes, so the twenty-minute spine is the first half and the interruptions are the second. If they do not interrupt, you use the spare time on failure and operations, not on a new box.

1. Minute 0 to 5. Assumptions, out of scope, one API, one database, server prices.
2. Minute 5 to 15. API codes, the four tables, the raw-body hash, the one transaction.
3. Minute 15 to 25. Relay, topic, two subscriptions, delivery key, complete after the provider accept.
4. Minute 25 to 35. The four panel answers: SQL role, rollback, second instance, poison message.
5. Minute 35 to 45. Dashboard, slot swap, the smoke that retries a key, and the list of things you refused. Stop. Ask them which assumption you should redraw.

Score yourself the way the roadmap scores a spoken answer. Correct: a working lead would agree with the transaction and the dedupe. Tradeoff: you named the microservice, the two-phase commit, and the in-memory cache as the things you did not ship. Consequence: the partner, the on-call person, or the customer feels a 409, a dead letter, or a swap. A correct drawing with no consequence is an incomplete mock. Do a second pairing on another evening if this one feels easy: the catalog and the realtime board, Days 3 and 9, with the same four questions. The SQL role, the swap, the second instance, and the poison message are not special to email. They fit any of these designs.

Interview trap

Redrawing six services because the mock is longer than twenty minutes. Answering the SQL-role question with the name of a cloud product and no permission. Promising a swap back after a destructive migration. Forgetting the second instance and leaving idempotency in a static field. Deleting the order because the SMS payload was bad. Inventing a quota when you run out of design.

Say this

The order, the idempotency row, and the outbox commit together. A topic fans out to email and SMS, and each consumer dedupes on the message id. The API login is not the migrator. I roll back by swapping a slot only while the old bits still match the schema. A second instance is safe because the unique keys are in SQL. A poison message dead-letters, and the order stays.

After the twenty days

Part A is done when you can write each pattern from a blank file and say the pitfall: missing key, overflow, unstable sort, recursion depth, off-by-one. Part B is done when you can redraw the order sheet and answer the four panel questions without this PDF. The roadmap is still the script for the leadership round. The deep dive is still the worked example when a follow-up goes into EF, SQL plans, or the service cut. The pipeline and MVC guide is still the request path. You do not reread all four the night before. You redraw the order sheet, you write two-sum and a binary search from memory, and you sleep.